Contents

1 Extra

2 Data Structures

2.1 BIT (Fenwick Tree)

Data structure that can handle point update and range queries for invertible operations. Obs: If the operation is not invertible (e.g. min,max,gcd), it can only answer prefix queries

```cpp
// [l,r] interval convention and 0-indexed parameters
struct BIT {
  vector<ll> bit;
  int n;
  BIT(int _n = 0){
    n = _n;
    bit.assign(n+1,0ll);
  }
  ll presum(int i){
    ll ans = 0;
    for (++i; i > 0; i -= i&-i)
      ans += bit[i];
    return ans;
  }
  ll sum(int l, int r){
    return presum(r) - presum(l-1);
  }
  void add(int i, ll v){
    for (++i; i <= n; i += i&-i)
      bit[i] += v;
  }
};
```

2.2 BIT (Fenwick Tree) 2D

2D Sum BIT, update and sum. The struct encapsulates the 1-based indexing, therefore parameters are 0-based indexed.

Query/update time: $\mathcal{O}(\log n \cdot \log m)$

Construction time (using proper constructor): $\mathcal{O}(nm)$

Space: $\mathcal{O}(nm)$

```cpp
// [l,r] interval cnvention and 0-based indexing
struct BIT2D {
  vector<vector<ll>> bit;
  int n, m;
  BIT2D(int _n, int _m){
    n = _n; m = _m;
    bit.assign(n+1,vector<ll>(m+1));
  }
  BIT2D(vector<vector<ll>> &a) {
```

```
    n = a.size();
    m = a[0].size();
    bit.assign(n+1, vector<ll>(m+1));

    for (int i = 1; i <= n; i++)
      for (int j = 1; j <= m; j++)
        bit[i][j] = a[i-1][j-1];

    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= m; j++) {
        int nxt = j + (j&-j);
        if (nxt <= m) bit[i][nxt] += bit[i][j];
      }
    }
    for (int i = 1; i <= n; i++) {
      int nxt = i + (i&-i);
      if (nxt <= n) {
        for (int j = 1; j <= m; j++)
          bit[nxt][j] += bit[i][j];
      }
    }
  }
  void add(int si, int sj, ll v){
    for(int i = si+1; i <= n; i+=i&-i)
      for (int j = sj+1; j <= m; j+=j&-j)
        bit[i][j] += v;
  }
  ll presum(int si, int sj){
    ll ans = 0;
    for (int i = si+1; i > 0; i-=i&-i)
      for (int j = sj+1; j > 0; j-=j&-j)
        ans += bit[i][j];
    return ans;
  }
  ll sum(ii ab, ii cd){
    auto [a,b] = ab;
    auto [c,d] = cd;
    return presum(c,d) - presum(a-1,d) - presum(c,b-1)
         + presum(a-1,b-1);
  }
};
```

2.3 DSU - Disjoint Set Union

Query/update time: $\mathcal{O}(1)$

Construction time: $\mathcal{O}(n)$

Space: $\mathcal{O}(n)$

```
struct DSU {
  vi p, sz;
  DSU(int n) {
    p.resize(n);
    iota(p.begin(), p.end(), 0);
    sz.assign(n, 1);
  }
  int find(int i) {
    if (p[i] == i) return i;
    return p[i] = find(p[i]);
  }
  bool join(int u, int v) {
    u = find(u);
    v = find(v);
    if (u == v) return false;
    if (sz[u] < sz[v]) swap(u, v);
    p[v] = u;
    sz[u] += sz[v];
    return true;
  }
};
```

2.4 DSU - Binary Tree

Specific code to find maximum path sums between pairs of vertices. Uses Kruskal-style MST. Query/update time: possibly $\mathcal{O}(n)$ Construction time: $\mathcal{O}(n)$ Space: $\mathcal{O}(n)$

```
vi d;
vi_ii e;
vi ans;

int merged;
vi _p, _leaf, _wei;
vvi adj;
int _find(int u) { return _p[u] == u ? u: _p[u] = _find
    (_p[u]); }
void _union(int u, int v, int w){
  u = _find(u);
  v = _find(v);
  int merge_ind = merged+n;
  _p[u] = merge_ind;
  _p[v] = merge_ind;
  _leaf[merge_ind] = _leaf[u] + _leaf[v];
  _wei[merge_ind] = max(_wei[u], _wei[v]);
  adj[u].push_back(merge_ind);
  adj[merge_ind].push_back(u);
  adj[v].push_back(merge_ind);
  adj[merge_ind].push_back(v);
  merged++;
}
void make(){
  _p = vi(2*n);
  for(int i = 0; i < 2*n; i++) _p[i] = i;
  _leaf = vi(2*n, 1);
  _wei = vi(2*n);
  for(int i = 0; i < n; i++) _wei[i] = d[i];
  merged = 0;
  adj = vvi(2*n);
}

void dfs(int u, int p){
  for(auto &v: adj[u]){
    if(v == p) continue;
    ans[v] = ans[u] + (_leaf[u] - _leaf[v])*_wei[u];
    dfs(v, u);
  }
}
```

2.5 MinQueue

Minimum (or maximum) queue. All operations are $\mathcal{O}(1)$ on average. Useful for sliding window max/min queries

```
template<typename T>
struct MinQueue{
  deque<pair<T,int>> q;
  int added = 0;
  int removed = 0;
  // returns [value,index]
  pair<T,int> getmin(){ return q.front(); }

  void push(T x){
    // < for MaxQueue
    while (!q.empty() && q.back().first > x) q.
        pop_back();
    q.push_back({x, added++});
  }
  void pop(){
    if (!q.empty() && q.front().second == removed) q.
        pop_front();
    removed++;
  }
  bool empty() { return added == removed; }
};
```

2.6 Mo's Algorithm

A technique for solving offline range queries on static arrays by sorting queries to minimize total pointer movement. It processes intervals by incrementally updating the range via `add` and `remove` operations. With the optimal block size, the time complexity is $\mathcal{O}((N+Q)\sqrt{N})$ or $\mathcal{O}(N\sqrt{Q})$, depending on block size choice ($\sqrt{N}$ or $N/\sqrt{Q}$). This example solves queries for distinct elements in range

```
struct Mo {
  struct Query {
    int l, r, idx, b;
    bool operator<(const Query& o) const {
      return b != o.b ? b < o.b :
        (b & 1 ? r > o.r : r < o.r);
    }
  };

  int n, block_sz;

  // custom stuff
  vi freq, a;
  int ans = 0;

  vector<Query> queries;
  Mo(int n) : n(n), block_sz(round(sqrt(n))) {}

  // [l,r] indexed
  void add_query(int l, int r, int i) {
    queries.push_back({l,r,i,l/block_sz});
  }
  void add(int i) {
    // add val at i
    freq[a[i]]++;
    if (freq[a[i]] == 1) ans++;
  }
  void remove(int i) {
    // remove value at i
    freq[a[i]]--;
    if (freq[a[i]] == 0) ans--;
  }
  int get_ans() {
    // compute current answer
    return ans;
  }

  vi run() {
    vi ans(queries.size());
    sort(queries.begin(), queries.end());
```

```
    int l = 0, r = -1;
    for (auto& q : queries) {
      while (l > q.l) add(--l);
      while (r < q.r) add(++r);
      while (l < q.l) remove(l++);
      while (r > q.r) remove(r--);
      ans[q.idx] = get_ans();
    }
    return ans;
  }
};
```

2.7 Mo with updates

Also sorts queries to minimize total pointer movement, but now there is an added dimmension for time (POINT updates). $\mathcal{O}(QN^{2/3})$

```
struct MoUpdate {
  int n;
  const int block_sz;
  struct Query {
    int l,r,t,id;
  };
  vector<Query> queries;
  vector<ii> updates;
  vector<int> a;

  MoUpdate(vector<int> &v, int q) : block_sz(round(cbrt
      (2*v.size()*v.size()))) {
    n = v.size() + q;
    a = v;
    init();
  }
  void add_query(int l, int r) {
    queries.push_back({l,r,(int)updates.size(),(int)
        queries.size()});
  }
  void add_update(int i, int x) {
    updates.push_back({i,x});
  }
  void update(int l, int r, int t) {
    auto &[i,v] = updates[t];
    if (l <= i && i <= r) remove(i);
    swap(a[i],v);
    if (l <= i && i <= r) add(i);
  }
  vector<int> run() {
    vector<int> ans(queries.size());
    sort(queries.begin(),queries.end(), [&](Query qa,
        Query qb){
      int l = qa.l/block_sz;
      int r = qa.r/block_sz;
      int ql = qb.l/block_sz;
      int qr = qb.r/block_sz;
      using iii = tuple<int,int,int>;
      return iii(l, l&1 ? -r:r, r&1 ? qa.t:-qa.t) <
        iii(ql, ql&1 ? -qr:qr, qr&1 ? qb.t:-qb.t);
    });
    int l = 0, r = -1, t = 0;
    for (auto &q : queries) {
      while(t < q.t) update(l,r,t++);
      while(t > q.t) update(l,r,--t);
      while(l > q.l) add(--l);
      while(r < q.r) add(++r);
      while(l < q.l) remove(l++);
      while(r > q.r) remove(r--);
      ans[q.id] = get_ans();
    }
    return ans;
  }

  // customize depending on problem
  int ans = 0;

  void init(){
    // initialize custom stuff
  }
  void remove(int i) {
    // remove a[i] from the current ans
  }
  void add(int i) {
    // add a[i] to the current ans
  }
  int get_ans() {}
};
```

2.8 Segment Tree

You should (almost) never alter the `segtree` logic. Ideally you only modify the `node` struct by answering four questions:

1. `tag`: What data does a range update carry? (e.g., a single integer for addition, or a pair of integers a, d for an AP).
2. `apply()`: How does a `tag` mathematically alter the state of a single node?
3. `push()`: How does a parent node transfer its pending `tag` to its `left` and `right` children?
4. `combine()`: How do you calculate the state of a parent by merging the states of its children?

OBS: lazy segtrees must be called with `has=true`

2.8.1 Iterative Point Update

Best for: Standard point updates and range queries (e.g., Point Add + Range Sum).

- **Performance:** Best possible.
- **Memory:** Best possible: $2n$ allocation.
- **Use Case:** If no range updates, then use this.

```
// ITERATIVE - implemented for sum + point assignment
struct segtree {
  struct node {
    int sum = 0; // data
    static node combine(const node& a, const node& b) {
      return {a.sum + b.sum};
    }
    void apply(int v) {
      sum = v;
    }
  };
  int n;
  vector<node> t;
  void init(const vi& a) {
    n = a.size();
    t.assign(2*n, node());
    for (int i = 0; i < n; i++) t[n + i].apply(a[i]);
    for (int i = n-1; i > 0; i--) t[i] = node::combine(
        t[i<<1], t[i<<1|1]);
  }
  void set(int p, int v) {
    for (t[p += n].apply(v), p >>= 1; p > 0; p >>= 1) {
      t[p] = node::combine(t[p<<1], t[p<<1|1]);
    }
  }
  node query(int l, int r) {
    node resL, resR;
    for (l+=n, r+=n; l < r; l>>=1, r>>=1) {
      if (l&1) resL = node::combine(resL, t[l++]);
      if (r&1) resR = node::combine(t[--r], resR);
    }
    return node::combine(resL, resR);
  }
};
```

2.8.2 Iterative Lazy

Best for: Index-independent range updates (e.g., Range Add, Range Set, Range Max, Range Flip).

- **Performance:** Noticeably faster than recursive lazy trees.
- **Index Blindness:** If the update depends on the range indexes themsleves (such as AP addition), use the recursive lazy one.

```
// LAZY ITERATIVE - implemented for Hash + range
    assignment
// for reference, see DoubleHashing implementation
// always call rangeUpdate with true on tag
struct segtree {
  struct node {
    struct tag {
      int val = 0;
      bool has = false;
    };
    Hash hash;
    int len = 0;
    tag lazy;
    static node combine(const node &a, const node &b){
      if (a.len == 0) return b;
      if (b.len == 0) return a;
      extend(a.hash.len);
      auto h1 = a.hash.h1 + pmod1[a.hash.len]*b.hash.h1
          ;
      auto h2 = a.hash.h2 + pmod2[a.hash.len]*b.hash.h2
          ;
      return {
        {h1,h2,a.hash.len+b.hash.len},
        a.len+b.len,
        {0,false}
      };
    }
    void apply(tag t){
      if (!t.has) return;
      // set hash as (t.val-offset)*sum

```

```
      if (hash.len > 0){
        extend(hash.len-1);
          hash.h1 = (t.val-OFFSET)*presumpmod1[hash.len-1];
          hash.h2 = (t.val-OFFSET)*presumpmod2[hash.len-1];
      }
      lazy = t;
    }
    void push(node &left, node &right){
      if (lazy.has){
        left.apply(lazy);
          right.apply(lazy);
          lazy = {0,false};
      }
    }
    void pull(const node &left, const node &right){
      tag cur = lazy;
      *this = combine(left,right);
      apply(cur);
    }
  };
  int n, h;
  vector<node> t;
  segtree(const string& a) {
    int sz = a.size();
    n = 1; h = 0;
    while (n < sz) { n*=2; h++; }

    t.assign(2*n, node());

    for (int i = 0; i < sz; i++) {
        t[i+n].len = 1;
        t[i+n].hash.len = 1;
        t[i+n].apply({a[i], true});
    }
    for (int i = n-1; i > 0; i--) t[i].pull(t[i<<1], t[i<<1|1]);
  }
  void build(int p) {
    while (p > 1) p >>= 1, t[p].pull(t[p<<1], t[p<<1|1]);
  }
  void push(int p) {
    for (int s = h; s > 0; s--) {
      int i = p >> s;
      t[i].push(t[i<<1], t[i<<1|1]);
    }
  }
  void rangeUpdate(int l, int r, node::tag v) {
    if (l==r) return;
    l += n, r += n;
    int l0 = l, r0 = r;

    push(l0); push(r0-1);
    for (; l < r; l >>= 1, r >>= 1) {
      if (l&1) t[l++].apply(v);
      if (r&1) t[--r].apply(v);
    }
    build(l0); build(r0-1);
  }
  node query(int l, int r) {
    if (l == r) return node();
    l += n, r += n;
    push(l); push(r-1);
    node resL = node(), resR = node();
    for (; l < r; l >>= 1, r >>= 1) {
      if (l&1) resL = node::combine(resL, t[l++]);
      if (r&1) resR = node::combine(t[--r], resR);
    }
    return node::combine(resL, resR);
  }
};
```

2.8.3 Recursive Lazy

Best for: Index-dependent updates (e.g. AP addition), Tree Walking (Binary Search on Tree), and persistency.

- **Performance:** It is the slowest of the three, use only if necessary.

```
// LAZY recursive - implemented for sum + range AP addition
struct segtree {
  struct node {
    struct tag {
      int a = 0, d = 0;
      bool has = false;
    };
    int sum = 0; // data
    tag lazy;
    static node combine(const node& left, const node& right) {
      return {left.sum + right.sum, {0, 0, false}};
    }
    void leaf(int v) { sum = v; }
    void apply(tag t, int lx, int rx) {
      if (!t.has) return;
      int len = rx - lx;

      // Sum of (t.a + t.d * i) for i in [lx, rx-1]
      sum += t.a * len + t.d*len*(lx+rx-1)/2;

      lazy.a += t.a;
      lazy.d += t.d;
      lazy.has = true;
    }
    void push(node& left, node& right, int lx, int rx) {
      if (lazy.has) {
        int mx = (lx + rx) / 2;
        left.apply(lazy, lx, mx);
        right.apply(lazy, mx, rx);
        lazy = {0, 0, false};
      }
    }
  };

  int size;
  vector<node> t;

  void init(int n) {
    size = 1;
    while (size < n) size *= 2;
    t.assign(2*size, node());
    build(0, 0, size, n);
  }
  void build(int x, int lx, int rx, int n) {
    if (rx-lx == 1) {
      if (lx < n) t[x].leaf(0);
      return;
    }
    int mx = (lx + rx) / 2;
    build(2*x+1, lx, mx, n);
    build(2*x+2, mx, rx, n);
    t[x] = node::combine(t[2*x+1], t[2*x+2]);
  }
  void push(int x, int lx, int rx) {
    t[x].push(t[2*x+1], t[2*x+2], lx, rx);
  }
  void rangeUpdate(int l, int r, node::tag v, int x, int lx, int rx) {
    if (lx >= r || rx <= l) return;
    if (lx >= l && rx <= r) {
      t[x].apply(v, lx, rx);
      return;
    }
    push(x, lx, rx);
    int mx = (lx + rx) / 2;
    rangeUpdate(l, r, v, 2*x+1, lx, mx);
    rangeUpdate(l, r, v, 2*x+2, mx, rx);
    t[x] = node::combine(t[2*x+1], t[2*x+2]);
  }
  void rangeUpdate(int l, int r, int a, int d) {
    rangeUpdate(l, r, {a - l*d, d, true}, 0, 0, size);
  }
  node query(int l, int r, int x, int lx, int rx) {
    if (lx >= r || rx <= l) return node();
    if (lx >= l && rx <= r) return t[x];
    push(x, lx, rx);
    int mx = (lx+rx) / 2;
    return node::combine(
      query(l, r, 2*x+1, lx, mx),
      query(l, r, 2*x+2, mx, rx)
    );
  }
  node query(int l, int r) {
    return query(l, r, 0, 0, size);
  }
};
```

2.9 Sparse Table RMQ

Sparse table for RMQ in $\mathcal{O}(1)$, used in many problems, including $\mathcal{O}(1)$ LCA (Trees) and LCP (SuffixArray) queries.

```
struct SparseTable {
  vector<vector<ii>> st;

  void build(const vi &a) {
    int n = a.size();
    int max_log = __lg(n)+1;
    st.assign(max_log, vector<ii>(n));
    for (int i = 0; i < n; i++) {
      st[0][i] = {a[i], i};
    }
    for (int i = 1; i < max_log; i++) {
      for (int j = 0; j + (1 << i) <= n; j++) {
        // Combine the two halves
        st[i][j] = f(st[i-1][j], st[i-1][j + (1 << (i-1))]);
      }
    }
  }

  // Returns min value and index in range [l, r] inclusive
  ii min(int l, int r) {
    int len = r - l + 1;
    int k = __lg(len);
    return f(st[k][l], st[k][r - (1<<k) + 1]);
  }

```

```
  inline ii f(ii l, ii r) { return std::min(l,r); }
};
```

3 Graphs

3.1 BFS 0-1

Time: $\mathcal{O}(n+m)$

```
vi bfs01(int s){
  vi d(n, INF);
  d[s] = 0;
  deque<int> q;
  q.push_front(s);
  while(!q.empty()){
    int u = q.front(); q.pop_front();
    for (auto [w,v] : adj[u]){
      if (d[u]+w < d[v]){
        d[v] = d[u] + w;
        if (w == 1) q.push_back(v);
        else q.push_front(v);
      }
    }
  }
  return d;
}
```

3.2 Block Cut Tree (BCT)

A data structure that condenses an undirected graph into a bipartite graph using two types of nodes: B-nodes (biconnected components) and C-nodes (articulation points). Edges strictly connect a B-node to a C-node if the articulation point belongs to that component. Yields exactly one tree per connected component.

Articulation points are explicitly represented as the C-nodes. Because an articulation point is a vertex that disconnects a component upon removal, it acts as a literal bridge in the BCT. Any path navigating between two distinct biconnected components in the original graph is forced to route through the shared C-node.

```
vvi build_bct(const vvi &adj){
  int n = adj.size();
  vi in(n,-1), low(n), st;
  int timer = 0;
  vvi bccs;

  auto dfs = [&](auto &&dfs, int u, int p) -> void {
    in [u] = low[u] = ++timer;
    st.push_back(u);
    if (p == -1 && adj[u].empty()){
      bccs.push_back({u});
      st.pop_back();
      return;
    }
    for (int v : adj[u]) if (v != p) {
      if (in[v] != -1) {
        low[u] = min(low[u], in[v]);
      } else {
        dfs(dfs,v,u);
        low[u] = min(low[u],low[v]);
        if (low[v] >= in[u]){
          bccs.push_back({u});
          while(1){
            int w = st.back();
            st.pop_back();
            bccs.back().push_back(w);
            if (w == v) break;
          }
        }
      }
    }
  };
  for (int i = 0; i < n; i++){
    if (in[i] == -1) dfs(dfs,i,-1);
  }
  vvi bct(n+bccs.size());
  for (int i = 0; i < bccs.size(); i++){
    int b = n+i;
    for (int u : bccs[i]){
      bct[u].push_back(b);
      bct[b].push_back(u);
    }
  }
  return bct;
}
```

3.3 Bridges and Articulation points

DFS to get bridges and articulation points of a graph in $\mathcal{O}(n+m)$

```
vvi adj;
vi in, low;
int timer;
set<int> cut_points;
vector<ii> bridges;

void dfs_ap(int u, int p = -1) {
  in[u] = low[u] = ++timer;
  int ch = 0;
  for (int v : adj[u]) {
    if (v == p) continue;
    if (in[v]) {
      // Back-edge
      low[u] = min(low[u], in[v]);
    } else {
      // Tree-edge
      dfs_ap(v, u);
      low[u] = min(low[u], low[v]);
      if (low[v] >= in[u] && p != -1)
        cut_points.insert(u);
      ch++;
    }
  }
  if (p == -1 && ch > 1)
    cut_points.insert(u);
}

void dfs_bridges(int u, int p = -1) {
  in[u] = low[u] = ++timer;
  for (int v : adj[u]) {
    if (v == p) continue;
    if (in[v]) {
      low[u] = min(low[u], in[v]);
    } else {
      dfs_bridges(v, u);
      low[u] = min(low[u], low[v]);
      if (low[v] > in[u])
        bridges.push_back({u, v});
    }
  }
}

void init(int n) {
  timer = 0;
  in.assign(n, 0);
  low.assign(n, 0);
  cut_points.clear();
  bridges.clear();
}
```

3.4 Dijkstra

Time: $\mathcal{O}(m \log n)$

```
void dijkstra(int s){
  int d, u, v;
  dist = vi(n, INF);
  dist[s] = 0;
  priority_queue<ii, vii, greater<ii>> pq;
  pq.emplace(0,s);
  while(!pq.empty()){
    auto [d,u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;
    for (auto &[w,v] : adj[u]){
      if (dist[v] > dist[u] + w){
        dist[v] = dist[u] + w;
        pq.emplace(dist[v], v);
      }
    }
  }
}
```

3.5 Dinic - Flow/matchings

- **General Network:** $\mathcal{O}(VE \log U)$.
- **Unit Capacity Network:** $\mathcal{O}(\min(V^{2/3}, E^{1/2}) \cdot E)$. Often considered $\mathcal{O}(E\sqrt{V})$.
- **Bipartite Matching:** $\mathcal{O}(\min(V^{2/3}, E^{1/2}) \cdot E)$. Often considered $\mathcal{O}(E\sqrt{V})$.

```
struct Dinic {
  struct Edge {
    int u, v;
    ll cap, flow = 0;
    Edge(int u, int v, ll cap) : u(u), v(v), cap(cap)
        {}
  };

  const ll flow_inf = 1e18;
  vector<Edge> edges;
  vvi adj;
  int n, m = 0;
  int s, t;
```

```
  vi level, ptr;
  queue<int> q;

  Dinic(int n): n(n) {
    adj.resize(n);
    level.resize(n);
    ptr.resize(n);
  }

  void add_edge(int u, int v, ll cap) {
    edges.emplace_back(u,v,cap);
    edges.emplace_back(v,u,0);
    adj[u].push_back(m++);
    adj[v].push_back(m++);
  }

  bool bfs(ll delta){
    queue<int> q;
    q.push(s);
    while(!q.empty()){
      int u = q.front(); q.pop();
      for (int id : adj[u]){
        auto &e = edges[id];
        if (e.cap - e.flow < delta) continue;
        if (level[e.v] != -1) continue;
        level[e.v] = level[u]+1;
        q.push(e.v);
      }
    }
    return level[t] != -1;
  }

  ll dfs(int u, ll pushed) {
    if (pushed == 0) return 0;
    if (u == t) return pushed;
    for (int &cid = ptr[u]; cid < (int)adj[u].size();
         cid++){
      int id = adj[u][cid];
      auto &e = edges[id];
      if (level[u]+1 != level[e.v]) continue;
      ll tr = dfs(e.v,min(pushed, e.cap - e.flow));
      if (tr == 0) continue;
      e.flow += tr;
      edges[id^1].flow -= tr;
      return tr;
    }
    return 0;
  }

  ll maxflow(int s, int t){
    this->s = s; this->t = t;
    ll max_c = 0;
    for (auto &e : edges) max_c = max(max_c, e.cap);

    ll delta = 1;
    while(delta <= max_c) delta <<= 1;
    delta >>= 1;

    ll f = 0;
    for (;delta > 0; delta >>= 1){
      while(true){
        fill(level.begin(), level.end(),-1);
        level[s] = 0;
        if (!bfs(delta)) break;
        fill(ptr.begin(), ptr.end(), 0);
        while(ll pushed = dfs(s,flow_inf)) f += pushed;
      }
    }
    return f;
  }


  // call constructor with (n1+n2+2) beforehand (dont
       add edges manually)
  // assumes pairs are 1-indexed
  vii maxmatchings(int n1, int n2, const vii& pairs){
    for (int i = 1; i <= n1; i++)
      add_edge(0,i,1);

    for (int i = 1; i <= n2; i++)
      add_edge(i+n1,n-1,1);

    for (auto &[u,v] : pairs)
      add_edge(u,v+n1,1);

    maxflow(0,n-1);

    vii matchings;
    for (auto &e : edges){
      if (e.u >= 1 && e.u <= n1 && e.flow == 1 && e.v >
           n1){
        matchings.emplace_back(e.u,e.v-n1);
      }
    }
    return matchings;
  }

  vii mincut(int s, int t){
    maxflow(s,t);
    queue<int> q; q.push(s);
    vector<bool> reachable(n);
    reachable[s] = true;
    while(!q.empty()){
      int u = q.front(); q.pop();
      for (auto &id : adj[u]){
        int v = edges[id].v;
        if (edges[id].cap - edges[id].flow > 0 && !
            reachable[v]) {
          reachable[v] = true;
          q.push(v);
        }
      }
    }

    vii minCutEdges;
    for (int i = 0; i < m; i += 2) {
      const Edge& edge = edges[i];
      if (reachable[edge.u] && !reachable[edge.v]) {
        minCutEdges.emplace_back(edge.u, edge.v);
      }
    }

    return minCutEdges;
  }
};
```

3.6 Floyd-Warshall

Time: $\mathcal{O}(n^3)$

```

vvi d(n, vi(n, INF));
void floyd_warshall(){
  for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        d[i][j] = min(d[i][j], d[i][k]+d[k][j]);
}
```

3.7 Hopcroft-Karp - Bipartite Matching

Bipartite matching such as Kuhn but faster. BFS until first layer missing match, DFS for the BFS graph to find pairings. Time: $\mathcal{O}(E\sqrt{V})$

```
int n, m, k;
vvi adj;
vi p, dist; /*p is in matching for [0, n[ and parent
     for [n, n+m[*/

int bfs(){
  queue<int> q;
  dist = vi(n+m, inf);
  for(int i = 0; i < n; i++){
    if(p[i] == -1) q.push(i), dist[i] = 0;
  }
  int min_dist_match = inf;
  while(!q.empty()){
    int u = q.front(); q.pop();
    if(dist[u] > min_dist_match) continue;
    for(auto v: adj[u]){
      if(p[v] == -1) min_dist_match = dist[u];
      else if(dist[p[v]] == inf){
        dist[p[v]] = dist[u] + 1;
        q.push(p[v]);
      }
    }
  }
  return min_dist_match != inf;
}

int dfs(int u){
  for(auto v: adj[u]){
    if(p[v] == -1 || (dist[u]+1 == dist[p[v]] && dfs(p[
         v]))){
      p[v] = u;
      p[u] = 1;
      return true;
    }
  }
  dist[u] = inf;
  return false;
}

int hopkarp(){
  p = vi(n+m, -1);
  int matchings = 0;
  while(bfs()){
    for(int i = 0; i < n; i++){
      if(p[i] == -1 && dfs(i)) matchings++;
    }
  }
  return matchings;
}

void create(){
  adj = vvi(n+m);
  for(int i = 0; i < k; i++){
    int u, v;
    cin >> u >> v; u--; v--;
    v += n;
    adj[u].push_back(v);
  }
}
```

3.8 Hungarian

Solves minimum cost assignment for n workers and m jobs. Time: $\mathcal{O}((n+m)^3)$

```
// cost should be (cost[worker][job])
pair<int,vii> hungarian(int n, int m, const vvi &cost) {
  if (n == 0) return {0,{}};
  int N = max(n, m);

  vi u(N+1), v(N+1), p(N+1), way(N+1);

  const int INF = 1e9;
  for (int i = 1; i <= n; ++i) {
    p[0] = i;
    int j0 = 0;
    vi minv(N + 1, INF);
    vector<bool> used(N + 1, false);

    do {
      used[j0] = true;
      int i0 = p[j0], delta = INF, j1;

      for (int j = 1; j <= N; ++j) {
        if (!used[j]) {
          int cur = cost[i0-1][j-1] - u[i0] - v[j];
          if (cur < minv[j]) {
            minv[j] = cur;
            way[j] = j0;
          }
          if (minv[j] < delta) {
            delta = minv[j];
            j1 = j;
          }
        }
      }

      for (int j = 0; j <= N; ++j) {
        if (used[j]) {
          u[p[j]] += delta;
          v[j] -= delta;
        } else {
          minv[j] -= delta;
        }
      }
      j0 = j1;
    } while (p[j0] != 0);

    do {
      int j1 = way[j0];
      p[j0] = p[j1];
      j0 = j1;
    } while (j0);
  }

  int total_cost = 0;
  for (int j = 1; j <= m; ++j) {
    if (p[j] != 0) {
      total_cost += cost[p[j] - 1][j - 1];
    }
  }

  // {worker,job}[] 0-indexed
  vii matchings;
  for (int j = 1; j <= m; ++j) {
    if (p[j] != 0) {
      matchings.push_back({p[j] - 1, j - 1});
    }
  }
  return {total_cost, matchings};
}
```

3.9 Kuhn - Bipartite Matching

Bipartite matching. Time: $\mathcal{O}(VE)$

```
int matchings;
vi p, vis;
vii match;

int dfs(int u){
  if(vis[u]) return 0;
  vis[u] = 1;
  for(auto v: adj[u]){
    if(p[v] == -1 || dfs(p[v])){
      p[v] = u;
      return 1;
    }
  }
  return 0;
}

void kuhn(){
  matchings = 0;
  p = vi(n+m, -1);
  for(int i = 0; i < n; i++){
    vis = vi(n, 0);
    matchings += dfs(i);
  }
  for(int i = n; i < n+m; i++){
    if(p[i] != -1) match.push_back(ii(p[i], i));
  }
}

void create(){
  adj = vvi(n+m);
  for(int i = 0; i < k; i++){
    int u, v;
    cin >> u >> v; u--; v--;
    adj[u].push_back(v+n);
  }
}
```

3.10 Min cost flow

Time: $\mathcal{O}(FE \log V)$

If negative costs are needed (maximize cost), need to run SPFA once at the start, making the solution $\mathcal{O}(EV + FE \log V)$.

```
struct MinCostFlow {
  struct Edge {
    int to, capacity, rev;
    ll cost;
  };

  int n;
  vector<vector<Edge>> adj;

  MinCostFlow(int _n) : n(_n), adj(_n) {}

  void add_edge(int from, int to, int cap, ll cost){
    adj[from].push_back({to,cap,(int)adj[to].size(), cost});
    adj[to].push_back({from,0,(int)adj[from].size()-1, -cost});
  }

  // O(FElog(V))
  lli min_cost_flow(int s, int t, int targetFlow) {
    int flow = 0;
    ll total_cost = 0;
    vll dist, h(n);
    vi pv, pe;

    // needed only if negative costs exists
    spfa(s, h, pv, pe);

    while (flow < targetFlow) {
      dijkstra(s, h, dist, pv, pe);

      if (dist[t] == INF) break;

      for (int i = 0; i < n; i++) {
        if (dist[i] < INF) {
          h[i] += dist[i];
        }
      }

      int f = targetFlow - flow;
      int cur = t;
      while (cur != s) {
        f = min(f, adj[pv[cur]][pe[cur]].capacity);
        cur = pv[cur];
      }

      flow += f;
      total_cost += f * h[t];
      cur = t;
      while (cur != s) {
        Edge &e = adj[pv[cur]][pe[cur]];
        e.capacity -= f;
        adj[e.to][e.rev].capacity += f;
        cur = pv[cur];
      }
    }


    return {total_cost, flow};
  }

  // needed only if negative costs exists
  void spfa(int s, vll &dist, vi &pv, vi &pe) {
    dist.assign(n, INF);
    pv.assign(n,-1);
    pe.assign(n,-1);
    vector<bool> inq(n, false);
    queue<int> q;

    dist[s] = 0;
    q.push(s);
    inq[s] = true;

    while (!q.empty()) {
      int u = q.front(); q.pop();
      inq[u] = false;
      for (int i = 0; i < adj[u].size(); i++) {
        Edge &e = adj[u][i];
        int v = e.to;
        if (e.capacity > 0 && dist[v] > dist[u] + e.cost) {
```

```
            dist[v] = dist[u] + e.cost;
            pv[v] = u;
            pe[v] = i;
            if (!inq[v]) {
              inq[v] = true;
              q.push(v);
            }
          }
        }
      }
    }

    void dijkstra(int s, vll &h, vll &dist, vi &pv, vi &pe) {
      dist.assign(n, INF);
      pv.assign(n, -1);
      pe.assign(n, -1);
      dist[s] = 0;

      priority_queue<lli, vector<lli>, greater<lli>> pq;
      pq.emplace(0, s);

      while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;

        for (int i = 0; i < adj[u].size(); i++) {
          Edge &e = adj[u][i];
          if (e.capacity <= 0) continue;
          int v = e.to;

          ll reduced_cost = e.cost + h[u] - h[v];
          if (dist[u] != INF && dist[v] > dist[u] + reduced_cost) {
            dist[v] = dist[u] + reduced_cost;
            pv[v] = u;
            pe[v] = i;
            pq.push({dist[v], v});
          }
        }
      }
    }
};

// usage
int nodes = 302; // amount of nodes in the network
MinCostFlow mcf(nodes);

for (int i = 0; i < 150; i++){
  mcf.add_edge(0, i+1, 1, 0); // source to node
  mcf.add_edge(i+151, nodes-1, 1, 0); // node to sink
}

for (int i = 0; i < n; i++){
  int a, b, c; cin >> a >> b >> c;
  mcf.add_edge(a, b+150, 1, -c); // edges in between (-c to maximize the cost)
}

// final max cost is -cost
auto [cost, flow] = mcf.min_cost_flow(0,nodes-1,150);
```

3.11 MST - Kruskal

Time: $\mathcal{O}(m \log m)$

```
vector<pair<int,ii>> edges; // [weight, (u,v)]
int kruskal(int n){
  int cost = 0;
  DSU dsu(n); // n is the numb of vertices
  sort(edges.begin(), edges.end());
  for (auto &[w,uv] : edges){
    auto [u,v] = uv;
    if (dsu.join(u,v)) cost += w;
  }
  return cost;
}
```

3.12 MST - Prim

Time: $\mathcal{O}(m \log n)$

```
vvii adj, mst;
vi taken;

int prim(){
    priority_queue<iii, vector<iii>, greater<iii>> pq;
    taken[0] = 1;
    for (auto [w,v] : adj[0]){
        if (!taken[v]) pq.push({w, {0,v}});
    }

    int cost = 0;
    while (!pq.empty()){
        auto [w,pu] = pq.top(); pq.pop();
        auto [p,u] = pu;
        if (!taken[u]) {
            cost += w;
            mst[p].emplace_back(w,u);
            mst[u].emplace_back(w,p);
            taken[u] = 1;
            for (auto [w,v] : adj[u]){
                if (!taken[v]) {
                    pq.push({w,{u,v}});
                }
            }
        }
    }
    return cost;
}
```

3.13 SCC compressing

Condensing a graph into a DAG through its strongly connected components can be useful for DP

```
struct SCCCondenser {
  int n, timer, scc_cnt;
  vi in, low, scc;
  stack<int> st;

  SCCCondenser(const vvi& adj) {
    n = adj.size();
    in.assign(n, 0);
    low.assign(n, 0);
    scc.assign(n, -1);
    timer = scc_cnt = 0;
    for (int i = 0; i < n; ++i)
      if (!in[i]) dfs(i, adj);
  }

  void dfs(int u, const vvi& adj) {
    in[u] = low[u] = ++timer;
    st.push(u);
    for (int v : adj[u]) {
      if (!in[v]) {
        dfs(v, adj);
        low[u] = min(low[u], low[v]);
      } else if (scc[v] == -1) {
        low[u] = min(low[u], in[v]);
      }
    }
    if (low[u] == in[u]) {
      while (true) {
        int v = st.top(); st.pop();
        scc[v] = scc_cnt;
        if (u == v) break;
      }
      scc_cnt++;
    }
  }

  // Returns {DAG, Aggregated Values}
  pair<vvi, vi> build(const vvi& adj, const vi& val) {
    vvi dag(scc_cnt);
    vi scc_val(scc_cnt);
    set<ii> edges;

    for (int u = 0; u < n; ++u) {
      scc_val[scc[u]] += val[u]; // Aggregate values
      for (int v : adj[u]) {
        if (scc[u] != scc[v]) {
          if (edges.count({scc[u], scc[v]})) continue;
          edges.insert({scc[u], scc[v]});
          dag[scc[u]].push_back(scc[v]);
        }
      }
    }
    return {dag, scc_val};
  }
};
```

3.14 Steiner Tree

The Steiner Tree problem finds the minimum-cost tree connecting a given set of **terminals** in a weighted graph. The solution may use non-terminal vertices to reduce the total cost.

- **DP State:** `dp[mask][u]` = min cost to connect the terminals in `mask` ending at `u`.
- **Time:** $\mathcal{O}(3^S \cdot N + 2^S \cdot M \log N)$

Requires non-negative edge weights. Problem is NP-hard

```
int p = dsu.find(special[0]);
for (int i : special){
  if (dsu.find(i) != p){
    cout << "impossivel\n"; return;
  }
}
vvi dp(1<<s, vi(n,INF));
for (int i = 0; i < s; i++)
  dp[1<<i][special[i]] = 0;

for (int mask = 0; mask < (1<<s); mask++){
  for (int sub = mask; sub > 0; sub = (sub-1)&mask){
    for (int u = 0; u < n; u++) {
```

```
      dp[mask][u] = min(dp[mask][u], dp[sub][u] + dp[
          mask^sub][u]);
    }
  }
  priority_queue<ii, vii, greater<ii>> pq;
  for (int u = 0; u < n; u++) pq.push({dp[mask][u], u})
      ;

  while(!pq.empty()){
    auto [d,u] = pq.top(); pq.pop();
    if (d > dp[mask][u]) continue;
    for (auto &[w,v] : adj[u]){
      if (dp[mask][v] > dp[mask][u] + w) {
        dp[mask][v] = dp[mask][u] + w;
        pq.push({dp[mask][v], v});
      }
    }
  }
}
```

4 DP

4.1 Bin Packing

Time: $\mathcal{O}(n \cdot 2^n)$ Space: $\mathcal{O}(2^n)$

```
vi w(n);

vector<ii> dp(1<<n, ii(INF,0));
// dp[i] = for the subset i(bitmask) (A,B) is the pair
    where
// A - the min number of knapsacks to store this subset
// B - the min size of a used knapsack

dp[0] = ii(0,INF);
for (int subset = 1; subset < (1<<n); subset++){
  for (int item = 0; item < n; item++){
    if (!((subset>>item)&1)) continue;
    int prevsubset = subset - (1<<item);
    ii prev = dp[prevsubset];

    if (prev.second + w[item] <= x) {
      // can fill the knapsack, fill it
      dp[subset] = min(dp[subset], ii(prev.first, prev.
          second+w[item]));
    } else {
      // cant fill the knapsack, create a new one
      dp[subset] = min(dp[subset], ii(prev.first+1, w[
          item]));
    }
  }
}

cout << dp[(1<<n)-1].first << endl;
```

4.2 Broken Profile DP

Solves the problem of counting how many ways to fill an nxm grid using 1x2 tiles. This technique can be used whenever the state dependence is only on the previous state (column). Time: $\mathcal{O}(mn2^n)$ Space: $\mathcal{O}(mn2^n)$

```
int dp[1002][12][1024];
dp[0][0][0] = 1;

for (int i = 0; i < m; i++){
  for (int j = 0; j < n; j++){
    for (int mask = 0; mask < (1<<n); mask++){
      if (mask & (1<<j)){
        int nxt_mask = mask - (1<<j);
        dp[i][j+1][nxt_mask] += dp[i][j][mask];
        dp[i][j+1][nxt_mask] %= M;
      } else {
        int q = mask + (1 << j);
        dp[i][j+1][q] += dp[i][j][mask];
        dp[i][j+1][q] %= M;
        if (j < n-1 && (mask & (1<<(j+1)))==0){
          q = mask + (1 << (j+1));
          dp[i][j+1][q] += dp[i][j][mask];
          dp[i][j+1][q] %= M;
        }
      }
    }
  }

  for (int p = 0; p < (1<<n); p++){
    dp[i+1][0][p] = dp[i][n][p];
  }
}
```

4.3 Convex Hull Trick (CHT)

- **Recurrence form:**

$$dp_i = \min_{j<i} \left(dp_j + (h_i - h_j)^2 + c\right)$$

Expanding:

$$dp_i = h_i^2 + c + \min_{j<i} \left((-2h_j)h_i + h_j^2 + dp_j\right)$$

Thus, for each j, insert the line

$$f_j(x) = (-2h_j)x + h_j^2 + dp_j$$

and query at $x = h_i$.

- **Slope monotonicity:** If coefficients a_j (slopes) are inserted in strictly decreasing (or increasing) order as j grows, and
- **Query monotonicity:** Values x_i for query come in non-decreasing (min) or increasing (max) order consistent with slope order,
- **Complexity:**
  - Insertion + amortized query in $\mathcal{O}(1)$ per operation (pointer walk) under monotonicity.
  - Non-monotonic case, generic CHT via binary search: $\mathcal{O}(\log n)$ per query.
  - General alternative: Li Chao Tree for insertions/queries in arbitrary order, $\mathcal{O}(\log M)$ per operation (where M is the domain of x).
- **Constraints:**
  - If it cannot be written in linear form, CHT does not apply.
  - If there is no monotonicity of slopes or queries, consider Li Chao Tree or CHT variant with binary search.

The example below solves the dp where the recurrence is:

$$dp_i = \min_{j<i} \left(dp_j + (h_i - h_j)^2 + c\right).$$

```
struct CHT {
  struct Line { // y = mx + c
    int m, c;
    Line(int m, int c) : m(m), c(c) {}
    int val(int x){
      return m*x + c;
    }
    int floorDiv(int num, int den) {
      if (den < 0) num = -num, den = -den;
      if (num >= 0) return num / den;
      else return - ( (-num + den - 1) / den );
    }
    int ceilDiv(int num, int den) {
      if (den < 0) num = -num, den = -den;
      if (num >= 0) return (num + den - 1) / den;
      else return - ( (-num) / den );
    }
    int intersect(Line l){
      // m1x + c1 = m2x + c2
      // x = (c2 - c1)/(m1 - m2)
      // if slopes are increasing, use floor div
      return ceilDiv(l.c - c, m - l.m);
    }
  };

  deque<pair<Line, int>> dq;

  void insert(int m, int c){
    Line newLine(m, c);
    if (!dq.empty() && newLine.m == dq.back().first.m)
        {
      // If slopes increasing, change to <=
      if (newLine.c >= dq.back().first.c) return;
      else dq.pop_back();
    }
    // if slopes increasing, change to <=
    while (dq.size() > 1 && dq.back().second >= dq.back
        ().first.intersect(newLine)){
      dq.pop_back();
    }
    if (dq.empty()){
```

```
      // assuming queries are positive numbers, may
          change to -INF or +INF if needed
      dq.emplace_back(newLine, 0);
      return;
    }
    dq.emplace_back(newLine, dq.back().first.intersect(
        newLine));
  }

  // dont use query and queryNonMonotonicValues in the
      same problem
  int query(int x){
    while (dq.size() > 1){
      // if slopes increasing, change to >=
      if (dq[1].second <= x) dq.pop_front();
      else break;
    }
    return dq[0].first.val(x);
  }


  int queryNonMonotonicValues(int x){
    int l=0, r=dq.size()-1, ans=0;
    while (l <= r) {
      int mid = (l+r)>>1;
      if (dq[mid].second <= x) {
        ans = mid;
        l = mid + 1;
      } else {
        r = mid - 1;
      }
    }
    return dq[ans].first.val(x);
  }
};

void solve(){
  int n, c; cin >> n >> c;
  vi h(n);
  for (auto &x : h) cin >> x;

  vi dp(n);
  dp[0] = 0;
  CHT cht;
  cht.insert(-2*h[0], h[0]*h[0]);
  for (int i = 1; i < n; i++){
    dp[i] = cht.query(h[i]) + c + h[i]*h[i];
    cht.insert(-2*h[i], h[i]*h[i] + dp[i]);
  }
  cout << dp[n-1] << endl;
}
```

4.4 Edit Distance (Levenshtein)

Very similar to LCS, in the sense that it considers prefixes already computed. Time: $\mathcal{O}(mn)$ Space: $\mathcal{O}(mn)$

```
vvi dp(n+1, vi(m+1));
for (int i = 0; i <= n; i++) dp[i][0] = i;
for (int i = 0; i <= m; i++) dp[0][i] = i;
for (int i = 1; i <= n; i++){
  for (int j = 1; j <= m; j++){
    dp[i][j] = min(
      min(dp[i][j-1]+1, dp[i-1][j]+1),
      dp[i-1][j-1] + (s[i-1]!=t[j-1])
    );
  }
}
```

4.5 Knapsack - 1D

Time: $\mathcal{O}(nW)$ Space: $\mathcal{O}(W)$

```
vi dp(W+1);
for (int i = 0; i < n; i++){
  for (int j = W; j >= w[i]; j--){
    dp[j] = max(dp[j], v[i] + dp[j-w[i]]);
  }
}
```

4.6 Knapsack - 2D

Time: $\mathcal{O}(nW)$ Space: $\mathcal{O}(nW)$

```
vvi dp(n+1,vi(W+1));
for (int c = 1; c <= W; c++){
  for (int i = 1; i <= n; i++){
    dp[i][c] = dp[i-1][w];
    if (c-w[i-1] >= 0) {
      dp[i][c] = max(dp[i][c], dp[i-1][c-w[i-1]] + v[i
          -1]);
    }
  }
}
```

4.7 LCS - Longest Common Subsequence

Subsequence generation included here. Time: $\mathcal{O}(mn)$ Space: $\mathcal{O}(mn)$

```
vvi dp(n+1,vi(m+1));

for (int i = 1; i <= n; i++){
  for (int j = 1; j <= m; j++){
    if (a[i-1] == b[j-1]) {
      dp[i][j] = dp[i-1][j-1]+1;
    } else if (dp[i][j-1] > dp[i-1][j]){
      dp[i][j] = dp[i][j-1];
    } else {
      dp[i][j] = dp[i-1][j];
    }
  }
}
```

4.8 LiChao Tree

Queries and insertions are all $\mathcal{O}(\log M)$. Where M is the size of the query interval the tree receives.

```
// Li Chao tree for minimum (or maximum) over domain [L
    , R].
// T should support +, *, comparisons.
// For integer x use eps = 0 and discrete mid+1
    splitting;
// For floating use eps > 0 and continuous splitting
    without +1.
template<typename T>
struct lichao_tree {
  // if max lichao, change to ::min()
  static const T identity = numeric_limits<T>::max();

  struct Line {
    T m, c;
    Line() {
      m = 0;
      c = identity;
    }
    Line(T m, T c) : m(m), c(c) {}
    T val(T x) { return m * x + c; }
  };

  struct Node {
    Line line;
    Node *lc, *rc;
    Node() : lc(0), rc(0) {}
  };

  T L, R, eps;
  deque<Node> buffer;
  Node* root;

  Node* new_node() {
    buffer.emplace_back();
    return &buffer.back();
  }

  lichao_tree() {}

  lichao_tree(T _L, T _R, T _eps) {
    init(_L, _R, _eps);
  }

  void clear() {
    buffer.clear();
    root = nullptr;
  }

  void init(T _L, T _R, T _eps) {
    clear();
    L = _L;
    R = _R;
    eps = _eps;
    root = new_node();
  }

  void insert(Node* &cur, T l, T r, Line line) {
    if (!cur) {
      cur = new_node();
      cur->line = line;
      return;
    }

    T mid = l + (r - l) / 2;
    if (r - l <= eps) return;

    // if max lichao, change to >
    if (line.val(mid) < cur->line.val(mid))
      swap(line, cur->line);

    // if max lichao, change to >
    if (line.val(l) < cur->line.val(l)) insert(cur->lc,
        l, mid, line);
    else insert(cur->rc, mid + 1, r, line);
  }

  T query(Node* &cur, T l, T r, T x) {
    if (!cur) return identity;

    T mid = l + (r - l) / 2;
    T res = cur->line.val(x);
```

```
    if (r - l <= eps) return res;

    // if max lichao, change min to max
    if (x <= mid) return min(res, query(cur->lc, l, mid, x));
    else return min(res, query(cur->rc, mid + 1, r, x));
  }

  void insert(T m, T c) { insert(root, L, R, Line(m, c)); }

  T query(T x) { return query(root, L, R, x); }
};
```

4.9 LIS - Longest Increasing Subsequence

Time: $\mathcal{O}(n \log n)$

```
// dp[i] := min value a subsequence of length i+1 can end in
// dp is an increasing sequence, allowing for bin search
int lis(const vi &a){
  vi dp;
  for (int x : a){
    auto it = lower_bound(all(dp), x);
    if (it == dp.end()) dp.push_back(x);
    else *it = x;
  }
  return dp.size();
}
```

4.10 SOSDP

```
int k; // amount of bits
vi a(1<<k);
// sosdp
for (int bit = 0; bit < k; bit++){
  for (int mask = 0; mask < (1<<k); mask++){
    if (mask>>bit&1) {
      a[mask] += a[mask ^ (1<<bit)];
    }
  }
}

// do stuff (such as multiplication for OR convolution)

// sosdp inverse
for (int bit = 0; bit < k; bit++){
  for (int mask = 0; mask < (1<<k); mask++){
    if (mask>>bit&1) {
      a[mask] -= a[mask ^ (1<<bit)];
    }
  }
}
```

4.11 Subset Sum

Almost identical to Knapsack, this code contains the subset reconstruction. Time: $\mathcal{O}(nS)$ Space: $\mathcal{O}(nS)$

```
vvi dp(n+1,vi(sum+1));

vvii p(n+1,vii(sum+1));

dp[0][0] = 1;

for (int i = 1; i <= n; i++){
  for (int s = 1; s <= sum; s++){
    if (s-a[i-1] >= 0 && dp[i-1][s-a[i-1]]){
      // sum is possible taking item i
      p[i][s] = {i-1,s-a[i-1]};
      dp[i][s] = 1;
    } else if (dp[i-1][s]) {
      // sum not possible taking item i
      // but still possible with other items (<i)
      p[i][s] = {i-1,s};
      dp[i][s] = 1;
    }
  }
}

if (!dp[n][target]) {
  cout << -1 << endl;
  return;
}

vi subset;
ii pos = {n,target};
while(pos != ii(0,0)){
  auto [i,s] = pos;
  if (p[i][s].second != s) subset.push_back(a[i-1]);
  pos = p[i][s];
}
```

4.12 Subset Sum with Bitset

If you only need reachability. Time: $\mathcal{O}(nS/w)$ (where w is sys word size) Space: $\mathcal{O}(S)$

```
vi a;
bitset<MAXSUM+1> dp; dp[0] = 1;
for (int x : a) dp |= (dp << x);
```

5 Trees

5.1 Centroid Decomposition

Recursively splits a tree by its center of mass, building a "Centroid Tree" with a strictly guaranteed $O(\log N)$ depth. Used for path queries where the tree is unrooted.

Build: $O(N \log N)$

Sample is solving "count distinct paths with number of edges between k_1 and k_2"

```
struct CentroidDecomposition{
  vvi t, ct;
  vi sz, added, cp;
  int n, root;
  CentroidDecomposition(const vvi &adj, int _k1, int _k2)
  {
    t = adj,
    n = adj.size();
    cp = sz = added = vi(n);
    root = -1;
    ct.resize(n);
    init(_k1, _k2);
    decompose(0,-1);
  }
  int dfs(int u, int p){
    sz[u] = 1;
    for (int v : t[u]){
      if (v == p || added[v]) continue;
      sz[u] += dfs(v,u);
    }
    return sz[u];
  }
  int centroid(int u, int size, int p = -1){
    for (int v : t[u]){
      if (v == p || added[v]) continue;
      if (2*sz[v] > size) return centroid(v,size,u);
    }
    return u;
  }
  void decompose(int u, int p){
    int size = dfs(u,p);
    int c = centroid(u,size);
    added[c] = 1;

    if (p == -1) root = c;
    else ct[p].push_back(c);
    cp[c] = p;

    process_centroid(c);

    for (int v : t[c])
      if (!added[v])
        decompose(v,c);
  }

  // custom stuff
  int k1, k2;
  ll ans;
  vi freq, distfreq;
  void init(int _k1, int _k2){
    ans = 0;
    k1 = _k1; k2 = _k2;
    distfreq = freq = vi(k2+1);
    freq[0] = 1;
  }
  void process_centroid(int c){
    int maxd = 0;
    int validsum = 0;
    for (int u : t[c]) if (!added[u]){
      distfreq[0] = 1;
      int mxd = 0;
      auto getdist = [&](auto &&getdist, int u, int p, int d) -> void {
        distfreq[d]++;
        mxd = max(mxd,d);
        if (d == k2) return;
        for (int v : t[u]){
          if (v == p || added[v]) continue;
          getdist(getdist,v,u,d+1);
        }
      }; getdist(getdist,u,c,1);
      int sum = validsum;
      for (int d = 1; d <= mxd; d++){
        ans += 1ll*sum*distfreq[d];
        sum -= freq[k2-d];
        if (k1-d-1 >= 0) sum += freq[k1-d-1];
```

```
      }
      for (int d = 1; d <= mxd; d++) {
        if (k1 <= d) validsum += distfreq[d];
        freq[d]+=distfreq[d], distfreq[d] = 0;
      }
      maxd = max(maxd,mxd);
    }
    for (int i = 1; i <= maxd; i++) freq[i] = 0;
  }
};
```

5.2 Sum of distances - rerooting

Given a tree, $f(u, v) :=$ distance from u to v in the tree, compute

$$\sum_{u,v} f(u, v)$$

. Time: $\mathcal{O}(n)$

```
vvi adj;
vi sum_going_down, sum_going_up, sz;

void dfs(int u, int p){
  for (auto v : adj[u]){
    if (v == p) continue;
    dfs(v,u);
    sz[u] += sz[v];
    sum_going_down[u] += sum_going_down[v];
  }
  sum_going_down[u] += sz[u];
}

void dfs2(int u, int p, int par_ans){
  int up_amount = sz[0] - sz[u];
  sum_going_up[u] += par_ans + up_amount;
  int sum = sum_going_down[u];
  for (auto v : adj[u]){
    if (v == p) continue;
    int par_amount = sz[0] - sz[v];
    // new p_ans is going up for is going up for p
    // + go up to u and go down to other subtrees
    dfs2(v,u, par_ans + par_amount + sum - (
        sum_going_down[v]+sz[v]));
  }
}

void solve(){
  adj = vvi(n);
  sum_going_down = sum_going_up = vi(n);
  sz = vi(n,1);
  // read adj
  dfs(0,0);
  dfs2(0,0,0);

  for (int i = 0; i < n; i++){
    cout << sum_going_down[i]+sum_going_up[i] << ' ';
  }
  cout << endl;
}
```

5.3 Edge HLD

Sometimes the value is on the edges, for this few things need to change, but here is a template. Pre-computation: $\mathcal{O}(n)$ Queries: $\mathcal{O}(\log^2 n)$

```

struct EdgeHLD {
  int n, timer = 0;
  vvii adj;
  vi parent, depth, size, heavy, head, value;
  vi tin, tout;

  segtree seg;

  void init(int _n, vvii& _adj) {
    n = _n;
    adj = _adj;
    value.assign(n, 0);
    parent.assign(n, -1);
    depth.assign(n, 0);
    size.assign(n, 0);
    heavy.assign(n, -1);
    head.assign(n, 0);
    tin.assign(n, 0);
    tout.assign(n, 0);
    timer = 0;

    // edgeWeight[v] = weight of edge (parent[v], v),
        for v>0
    // root (0) has no parent, so its value is dummy
        (0)

    dfs1(0,0,0);
    dfs2(0, 0);

    vi linear(n);
    for (int u = 0; u < n; u++)
      linear[tin[u]] = value[u]; // position stores
          edge weight

    seg.init(linear);
  }

  int dfs1(int u, int p, int w) {
    size[u] = 1;
    parent[u] = p;
    value[u] = w;
    int max_sz = 0;
    for (auto[v,w] : adj[u]) {
      if (v == p) continue;
      depth[v] = depth[u] + 1;
      int sz = dfs1(v, u, w);
      size[u] += sz;
      if (sz > max_sz) {
        max_sz = sz;
        heavy[u] = v;
      }
    }
    return size[u];
  }

  void dfs2(int u, int h) {
    tin[u] = timer++;
    head[u] = h;
    if (heavy[u] != -1)
      dfs2(heavy[u], h);
    for (auto [v,w] : adj[u]) {
      if (v != parent[u] && v != heavy[u])
        dfs2(v, v);
    }
    tout[u] = timer;
  }

  // u deve ser o filho
  void update_edge(int u, int val) {
    seg.set(tin[u], val);
  }

  void rangeUpdate(int u, int v, int x) {
    while (head[u] != head[v]) {
      if (depth[head[u]] < depth[head[v]]) swap(u, v);
      seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
      u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    seg.rangeUpdate(tin[u] + 1, tin[v] + 1, x); // +1
        to skip LCA's edge
  }

  void update_subtree(int u, int x) {
    // updates all edges in subtree of u (skip incoming
        edge to u)
    seg.rangeUpdate(tin[u] + 1, tout[u], x);
  }

  segtree::node query(int u, int v) {
    segtree::node res = seg.NEUTRAL;
    while (head[u] != head[v]) {
      if (depth[head[u]] < depth[head[v]]) swap(u, v);
      res = seg.merge(res, seg.query(tin[head[u]], tin[
          u] + 1));
      u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    res = seg.merge(res, seg.query(tin[u] + 1, tin[v] +
        1)); // skip LCA's edge
    return res;
  }

  segtree::node query_subtree(int u) {
    // query all edges in subtree of u
    return seg.query(tin[u] + 1, tout[u]);
  }
};
```

5.4 HLD - Heavy light decomposition

If you need to compute a function on a path in a tree and need to support value updates on nodes, HLD is the way. Pre-computation: $\mathcal{O}(n)$ Queries: $\mathcal{O}(\log^2 n)$

OBS: this implementation uses the same segtree as this notebook, with 0-indexing and open-closed interval convention. Ideally, just change the segtree to change the computed function, the HLD struct remains the same. OBS2: this template also supports mass updates (path/subtree) and subtree queries.

```
struct HLD {
  int n, timer = 0;
  vvi adj;
  vi parent, depth, size, heavy, head, value;
  vi tin, tout;

  segtree seg;

```

```
  void init(int _n, vi& _value, vvi& _adj) {
    n = _n;
    adj = _adj;
    value = _value;
    parent.assign(n, -1);
    depth.assign(n, 0);
    size.assign(n, 0);
    heavy.assign(n, -1);
    head.assign(n, 0);
    tin.assign(n, 0);
    tout.assign(n, 0);
    timer = 0;

    dfs1(0);
    dfs2(0, 0);

    vi linear(n);
    for (int u = 0; u < n; u++)
      linear[tin[u]] = value[u];

    seg.init(linear);
  }

  int dfs1(int u) {
    size[u] = 1;
    int max_sz = 0;
    for (int v : adj[u]) {
      if (v == parent[u]) continue;
      parent[v] = u;
      depth[v] = depth[u] + 1;
      int sz = dfs1(v);
      size[u] += sz;
      if (sz > max_sz) {
        max_sz = sz;
        heavy[u] = v;
      }
    }
    return size[u];
  }

  void dfs2(int u, int h) {
    tin[u] = timer++;
    head[u] = h;
    if (heavy[u] != -1)
      dfs2(heavy[u], h);
    for (int v : adj[u]) {
      if (v != parent[u] && v != heavy[u])
        dfs2(v, v);
    }
    tout[u] = timer;
  }

  void update(int u, int val) {
    seg.set(tin[u], val);
  }

  void rangeUpdate(int u, int v, int x) {
    while (head[u] != head[v]) {
      if (depth[head[u]] < depth[head[v]]) swap(u, v);
      seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
      u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    seg.rangeUpdate(tin[u], tin[v] + 1, x);
  }

  void update_subtree(int u, int x) {
    seg.rangeUpdate(tin[u], tout[u], x);
  }

  segtree::node query(int u, int v) {
    segtree::node res = seg.NEUTRAL;
    while (head[u] != head[v]) {
      if (depth[head[u]] < depth[head[v]])
        swap(u, v);
      res = seg.merge(res, seg.query(tin[head[u]], tin[
          u]+1));
      u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    res = seg.merge(res, seg.query(tin[u], tin[v]+1));
    return res;
  }

  segtree::node query_subtree(int u){
    return seg.query(tin[u], tout[u]);
  }
};
```

5.5 LCA - RMQ

Pre-computation: $\mathcal{O}(n \log n)$

Queries: $\mathcal{O}(1)$

```
struct LCARMQ {
  SparseTable st;
  vi tin, dep, et_nodes, et_deps;

  LCARMQ(vvi &adj){
    int n = adj.size(), timer = 0;
    tin = dep = vi(n);
    et_nodes.reserve(2*n);
    et_deps.reserve(2*n);

    auto dfs = [&](auto &&dfs, int u, int p) -> void {
      et_nodes.push_back(u);
      et_deps.push_back(dep[u]);
      tin[u] = timer++;
      for (int v : adj[u]) if (v != p) {
        dep[v] = dep[u]+1;
        dfs(dfs,v,u);
        et_nodes.push_back(u);
        et_deps.push_back(dep[u]);
      }
      timer++;
    };
    dfs(dfs,0,0);

    st.build(et_deps);
  }

  int get(int u, int v){
    int tu = tin[u], tv = tin[v];
    if (tu > tv) swap(tu,tv);
    auto [val,id] = st.min(tu,tv);
    return et_nodes[id];
  }
};
```

5.6 LCA - binary lifting

Pre-computation: $\mathcal{O}(n \log n)$ Queries: $\mathcal{O}(\log n)$ OBS: just call `dfs(root)` before starting queries.

```
vvi adj, up;
vi tin, tout;
int timer = 0;

void dfs(int u, int p){
  tin[u] = timer++;
  for (auto v : adj[u]){
    if (v == p) continue;
    up[v][0] = u;
    for (int dist = 1; dist < LOGN; dist++){
      up[v][dist] = up[up[v][dist-1]][dist-1];
    }
    dfs(v);
  }
  tout[u] = timer++;
}

int isAncestor(int u, int v){
  return tin[u] <= tin[v] && tout[v] <= tout[u];
}

int lca(int u, int v){
  if (isAncestor(u,v)) return u;
  if (isAncestor(v,u)) return v;
  for (int dist = LOGN-1; dist >= 0; dist--){
    if(!isAncestor(up[u][dist],v)) u = up[u][dist];
  }
  return up[u][0];
}
```

6 Problemas clássicos

6.1 2SAT

Struct for solving 2SAT problems that supports many types of boolean expressions. To add a negated literal use u

```
// para adicionar negacao usar ~u
// Ex: a clausula (a v !b) se traduz para add_or(a,~b)
struct TwoSatSolver {
  int n;
  vvi adj, adjT;
  vector<bool> vis, assignment;
  vi topo, scc;

  void build(int _n){
    n = 2*_n;
    adj.assign(n,vi());
    adjT.assign(n,vi());
  }

  int get(int u){
    if (u < 0) return 2*(~u)+1;
    else return 2*u;
  }

  // u -> v
  void add_impl(int u, int v){
    u = get(u), v = get(v);
    adj[u].push_back(v);
    adjT[v].push_back(u);
    adj[v^1].push_back(u^1);
    adjT[u^1].push_back(v^1);
  }

  // u || v
```

```
  void add_or(int u, int v){
    add_impl(~u, v);
  }

  // u && v
  void add_and(int u, int v){
    add_or(u,u); add_or(v,v);
  }

  // u ^ v (equiv of x != v)
  void add_xor(int u, int v){
    add_impl(u, ~v);
    add_impl(~u, v);
  }

  // u == v
  void add_equals(int u, int v){
    add_impl(u, v);
    add_impl(v, u);
  }

  void toposort(int u){
    vis[u] = true;
    for (int v : adj[u])
      if (!vis[v]) toposort(v);
    topo.push_back(u);
  }

  void dfs(int u, int c){
    scc[u] = c;
    for (int v : adjT[u])
      if (!scc[v]) dfs(v,c);
  }

  pair<bool, vector<bool>> solve(){
    topo.clear();
    vis.assign(n, false);

    for (int i = 0; i < n; i++)
      if (!vis[i]) toposort(i);

    reverse(topo.begin(), topo.end());

    scc.assign(n, 0);
    int c = 0;
    for (int u : topo)
      if (!scc[u]) dfs(u,++c);

    assignment.assign(n/2, false);
    for (int i = 0; i < n; i += 2){
      if (scc[i] == scc[i+1]) return {false, {}};
      assignment[i/2] = scc[i] > scc[i+1];
    }

    return {true, assignment};
  }
};
```

6.2 Next Greater Element

One of the classic stack applications. Easy to translate to lower, leq or geq, just change the comparator of the `while`.

```
vi ngt(n, n);
stack<ii> st;
for (int i = 0; i < n; i++){
  while (!st.empty() && st.top().first < h[i]){
    ngt[st.top().second] = i;
    st.pop();
  }
  st.emplace(h[i], i);
}
```

7 Strings

7.1 Aho-Corasick

```
const int MAX = 1e6;
namespace aho {
  int next[MAX][26];
  int link[MAX], sz = 0, freq[MAX];
  vi leaves[MAX], order;
  void insert(const string& pat, int id) {
    int u = 0;
    for (char c : pat) {
      int x = c - 'a';
      if (!next[u][x]) next[u][x] = ++sz;
      u = next[u][x];
    }
    leaves[u].push_back(id);
  }
  void build() {
    queue<int> q; q.push(0);
    link[0] = -1;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      order.push_back(u);
      for (int x = 0; x < 26; x++) {
        int v = next[u][x];
        if (!v) continue;
        int l = link[u];
        while (l != -1 && !next[l][x]) l = link[l];
        link[v] = l == -1 ? 0 : next[l][x];
        q.push(v);
      }
    }
  }
  vi query(const string& txt, int num_patterns) {
    vi ans(num_patterns);
    int u = 0;
    for (char c : txt){
      int x = c - 'a';
      while (u != -1 && !next[u][x]) u = link[u];
      u = u == -1 ? 0 : next[u][x];
      freq[u]++;
    }
    for (int i = (int)order.size() - 1; i >= 0; i--) {
      int u = order[i];
      if (link[u] != -1) freq[link[u]] += freq[u];
    }
    for (int i = 1; i <= sz; i++)
      for (int id : leaves[i])
        ans[id] += freq[i];
    return ans;
  }
}
```

7.2 Hashing

Creation time: $\mathcal{O}(n)$ Access time: $\mathcal{O}(1)$ Space: $\mathcal{O}(n)$

```
namespace DoubleHashing {
  const int MOD1 = 2147483647, MOD2 = 2147483629;
  using mint1 = mint<MOD1>;
  using mint2 = mint<MOD2>;
  vector<mint1> pmod1{1}, invpmod1{1};
  vector<mint2> pmod2{1}, invpmod2{1};

  const int MAX_VAL = 256;
  const int OFFSET = 'a'-1;
  mt19937 rng(chrono::steady_clock::now().
      time_since_epoch().count());
  const mint1 p1 = uniform_int_distribution<int>(
      MAX_VAL+1, MOD1-1)(rng);
  const mint2 p2 = uniform_int_distribution<int>(
      MAX_VAL+1, MOD2-1)(rng);
  const mint1 inv1 = mint1::inv(p1);
  const mint2 inv2 = mint2::inv(p2);

  void extend(int n){
    while(pmod1.size() <= n){
      pmod1.push_back(pmod1.back()*p1);
      pmod2.push_back(pmod2.back()*p2);
      invpmod1.push_back(invpmod1.back()*inv1);
      invpmod2.push_back(invpmod2.back()*inv2);
    }
  }
  struct Hash {
    mint1 h1;
    mint2 h2;
    int len = 0;
    Hash apply(int x) const {
      extend(len+1);
      return {
        h1+pmod1[len]*(x-OFFSET),
        h2+pmod2[len]*(x-OFFSET),
        len+1
      };
    }
    bool operator<(const Hash &o) const {
      if (h1.val != o.h1.val) return h1.val < o.h1.val;
      if (h2.val != o.h2.val) return h2.val < o.h2.val;
      return len < o.len;
    }
    bool operator==(const Hash &o) const {
      return h1.val == o.h1.val && h2.val == o.h2.val
          && len == o.len;
    }
  };

  struct PrefixHash {
    vector<Hash> pref;
    PrefixHash(const string &s){
      int n = s.length();
      pref.reserve(n+1);
      pref.emplace_back();
      for (auto c : s)
        pref.push_back(pref.back().apply(c));
    }
    Hash get(int l, int r) const {
      l++,r++;
      mint1 h1 = (pref[r].h1 - pref[l-1].h1)*invpmod1[l
          -1];
      mint2 h2 = (pref[r].h2 - pref[l-1].h2)*invpmod2[l
          -1];
      return {h1,h2,r-l+1};
    }
  };
};
```

7.3 KMP

```cpp
vi compute_lps(const string &pat){
  int m = pat.length();
  vi lps(m);
  int len = 0;
  for (int i = 1; i < m; i++){
    while(len > 0 && pat[i] != pat[len])
      len = lps[len-1];
    if (pat[i] == pat[len]) len++;
    lps[i] = len;
  }
  return lps;
}

// find all occurrences
vi kmp_search(const string &txt, const string &pat){
  int n = txt.length();
  int m = pat.length();
  if (m == 0) return {};
  vi lps = compute_lps(pat);
  vi occurrences;
  int j = 0;
  for (int i = 0; i < n; i++){
    while (j > 0 && txt[i] != pat[j])
      j = lps[j-1];
    if (txt[i] == pat[j]) j++;

    if (j == m) {
      occurrences.push_back(i-m+1);
      j = lps[j-1];
    }
  }
  return occurrences;
}

// find all occurrences (simpler version)
vi kmp_search(const string &txt, const string &pat){
  int n = txt.length(), m = pat.length();
  vi lps = compute_lps(pat + '#' + txt);
  vi occurrences;
  for (int i = 0; i < n+m+1; i++){
    if (lps[i] == pat.length())
      occurrences.push_back(i-m*2);
  }
  return occurrences;
}

// borda sao os prefixos que tambem sao sufixos
vi find_borders(const string &s){
  vi lps = compute_lps(s);
  int i = s.length()-1;

  vi ans;
  while (lps[i] > 0){
    ans.push_back(lps[i]);
    i = lps[i]-1;
  }
  reverse(ans.begin(), ans.end());
  return ans;
}
```

7.4 Suffix Array

Time: $\mathcal{O}(n \log n)$ Space: $\mathcal{O}(n)$

```cpp
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(vi s, int lim = 256) {
    s.push_back(0);
    int n = s.size(), k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1ll, 2 * j),
         lim = p) {
      p = j, iota(all(y), n - j);
      for (int i = 0; i < n; i++)
        if (sa[i] >= j)
          y[p++] = sa[i] - j;
      fill(all(ws), 0);
      for (int i = 0; i < n; i++) ws[x[i]]++;
      for (int i = 1; i < lim; i++) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      for (int i = 1; i < n; i++) {
        a = sa[i - 1], b = sa[i];
        x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ?
             p - 1 : p++;
      }
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1]; s[i + k] == s[j
           + k]; k++);
  }
};
```

7.5 Suffix Automaton

```cpp
#define ALPHABET 26
struct SAM {
  struct State {
    int len = 0, link = -1;
    int cnt = 0, first_occ = -1;
    int next[ALPHABET];
    State() { fill(next, next + ALPHABET, -1); }
  };
  vector<State> st;
  int last;
  SAM(const string &s) {
    st.reserve(2*s.length());
    st.push_back(State());
    last = 0;
    for (int i = 0; i < s.length(); i++) extend(s[i], i
         );
    calc_cnt();
  }
  void extend(char c, int id) {
    int cur = st.size();
    st.push_back(State());
    st[cur].len = st[last].len + 1;
    st[cur].first_occ = id;
    st[cur].cnt = 1;
    int p = last;
    while (p != -1 && st[p].next[c-'a'] == -1) {
      st[p].next[c-'a'] = cur;
      p = st[p].link;
    }
    if (p == -1) {
      st[cur].link = 0;
      last = cur;
      return;
    }
    int q = st[p].next[c - 'a'];
    if (st[p].len + 1 == st[q].len) {
      st[cur].link = q;
      last = cur;
      return;
    }
    int clone = st.size();
    st.push_back(st[q]);
    st[clone].len = st[p].len + 1;
    st[clone].cnt = 0;
    while (p != -1 && st[p].next[c - 'a'] == q) {
      st[p].next[c - 'a'] = clone;
      p = st[p].link;
    }
    st[q].link = st[cur].link = clone;
    last = cur;
  }
  void calc_cnt() {
    int sz = st.size();
    int mx = 0;
    for (auto &s : st) mx = max(mx, s.len);

    vi c(mx+1);
    vi nodes(sz);
    for (int i = 0; i < sz; i++) c[st[i].len]++;
    for (int i = 1; i <= mx; i++) c[i] += c[i-1];
    for (int i = 0; i < sz; i++) nodes[--c[st[i].len]]
         = i;

    for (int i = sz-1; i >= 0; i--) {
      int u = nodes[i];
      if (st[u].link != -1) st[st[u].link].cnt += st[u
           ].cnt;
    }
  }
  string longest_common_substring(const string &t) {
    int v = 0, l = 0, len = 0, pos = 0;
    for (int i = 0; i < t.length(); i++) {
      int c = t[i] - 'a';
      while (v != 0 && st[v].next[c] == -1) {
        v = st[v].link;
        l = st[v].len;
      }
      if (st[v].next[c] != -1) {
        v = st[v].next[c];
        l++;
      }
      if (l > len) {
        len = l;
        pos = i;
      }
    }
    if (len == 0) return "";
    return t.substr(pos-len+1, len);
  }
  int count_occurrences(const string &t){
    int cur = 0;
    for (char c : t){
      if (st[cur].next[c-'a'] == -1) return 0;
      cur = st[cur].next[c-'a'];
    }
    return st[cur].cnt;
  }
  int first_occurrence(const string &t){
    int cur = 0;
    for (char c : t){
      if (st[cur].next[c-'a'] == -1) return -1;
      cur = st[cur].next[c-'a'];
    }
    return st[cur].first_occ-t.length()+1;
  }
  ll distinct_substrings(){
    ll ans = 0;
    for (int i = 1; i < st.size(); i++)
      ans += st[i].len - st[st[i].link].len;
    return ans;
  }
```

```
vector<ll> distinct_substrings_perlen(int n){
  vector<ll> diff(n+2);
  for (int i = 1; i < st.size(); i++){
    int l = st[st[i].link].len+1;
    int r = st[i].len;
    diff[l]++; diff[r+1]--;
  }
  vector<ll> ans(n+1);
  ans[0] = diff[0];
  for (int i = 1; i <= n; i++){
    ans[i] = ans[i-1]+diff[i];
  }

  return ans;
}
vector<ll> dp;
void calc_paths(int u){
  if (dp[u] != -1) return;
  dp[u]=1;
  for (int c = 0; c < ALPHABET; c++){
    int v = st[u].next[c];
    if (v == -1) continue;
    calc_paths(v);
    dp[u] += dp[v];
  }
}
string find_kth(ll k) {
  dp.assign(st.size(), -1);
  calc_paths(0);
  int u = 0;
  string ans = "";
  while (k > 0) {
    for (int c = 0; c < ALPHABET; c++) {
      int v = st[u].next[c];
      if (v == -1) continue;
      if (k <= dp[v]) {
        ans += 'a' + c;
        u = v;
        k--;
        break;
      } else k -= dp[v];
    }
  }
  return ans;
}
void calc_paths_with_repetitions(int u){
  if (dp[u] != -1) return;
  dp[u]=st[u].cnt;
  for (int c = 0; c < ALPHABET; c++){
    int v = st[u].next[c];
    if (v==-1) continue;
    calc_paths_with_repetitions(v);
    dp[u] += dp[v];
  }
}
string find_kth_with_repetitions(ll k){
  dp.assign(st.size(),-1);
  calc_paths_with_repetitions(0);
  int u = 0;
  string ans = "";
  while(k>0){
    for (int c = 0; c < ALPHABET; c++){
      int v = st[u].next[c];
      if (v==-1) continue;
      if (k <= dp[v]) {
        ans += 'a' + c;
        u = v;
        k -= st[v].cnt;
        break;
      } else k -= dp[v];
    }
  }
  return ans;
  }
};
```

7.6 Z

$$z[i] := max(k)|s[0..k-1] = s[i..i+k-1]$$

Time: $\mathcal{O}(n+m)$ Space: $\mathcal{O}(n+m)$

```
vi compute_z(const string &s) {
    int n = s.length();
    vi z(n);
    int l = 0, r = 0;

    for (int i = 1; i < n; i++) {
        if (i <= r)
            z[i] = min(r - i + 1, z[i-l]);

        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }

    return z;
}

vi find_occurrences(const string &txt, const string &
    pat){
    vi occurences;
    vi z = compute_z(pat + '#' + txt);
    int n = txt.length(), m = pat.length();
    for (int i = 0; i < n+m+1; i++){
        if (z[i] == m) occurences.push_back(i-m-1);
    }
    return occurences;
}
```

8 Math

8.1 Combinatorics (Pascal's Triangle)

Computes "n choose k". Requires factorials to be precomputed. Time: $\mathcal{O}(\log ZAP)$

8.1.1 Combinatorial Analysis

Fundamental Counting Principles

- **Permutations:** The number of ways to arrange k items from a set of n distinct items.

$$P(n,k) = \frac{n!}{(n-k)!}$$

- **Combinations (Binomial Coefficient):** The number of ways to choose k items from a set of n distinct items, regardless of order.

$$\binom{n}{k} = C(n,k) = \frac{n!}{k!(n-k)!}$$

- **Combinations with Repetition (Stars and Bars):** The number of ways to choose k items of n types, allowing repetitions. Equivalently, the number of ways to distribute k identical balls into n distinct urns.

$$\binom{k+n-1}{n-1} = \binom{k+n-1}{k}$$

Binomial Coefficient Properties and Pascal's Triangle

- **Pascal's Triangle**

$\binom{0}{0}$

$\binom{1}{0}\binom{1}{1}$

$\binom{2}{0}\binom{2}{1}\binom{2}{2}$

$\binom{3}{0}\binom{3}{1}\binom{3}{2}\binom{3}{3}$

$\binom{4}{0}\binom{4}{1}\binom{4}{2}\binom{4}{3}\binom{4}{4}$

- **Stifel's Relation:** Each element in Pascal's Triangle is the sum of the two elements immediately above it.

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

- **Symmetry:** Elements of a row are symmetric with respect to the center. Choosing k elements is the same as choosing the $n-k$ elements to be left behind.

$$\binom{n}{k} = \binom{n}{n-k}$$

- **Row Sum:** The sum of all elements in row n of Pascal's Triangle (where the first row is $n=0$) is equal to 2^n.

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n$$

- **Hockey Stick Identity:** The sum of elements in a diagonal, starting at

$$\binom{r}{r}$$

and ending at

$$\binom{n}{r}$$

, is equal to the element in the next row and next column,

$$\binom{n+1}{r+1}$$

.

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$$

- **Binomial Theorem:**

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k$$

- **Vandermonde's Identity:**

$$\sum_{j=0}^{k} \binom{m}{j}\binom{n}{k-j} = \binom{m+n}{k}$$

The easiest way to understand the identity is through a counting problem. Imagine you have a committee with m men and n women. How many ways can you form a subcommittee of k people?

Way 1 Direct Counting

You have a total of m+n people and need to choose k of them. The number of ways to do this is simply:

$$\binom{m+n}{k}$$

Way 2 Counting by Cases

We can divide the problem into cases, based on how many men (j) are chosen for the subcommittee.

Case 0: Choose 0 men and k women. The number of ways is

$$\binom{m}{0}\binom{n}{k}$$

.

Case 1: Choose 1 man and k-1 women. The number of ways is

$$\binom{m}{1}\binom{n}{k-1}$$

.

Case j: Choose j men and $k-j$ women. The number of ways is

$$\binom{m}{j}\binom{n}{k-j}$$

.

Other Important Concepts

- **Catalan Numbers:** A sequence of natural numbers that occurs in various counting problems (e.g., number of binary trees, balanced parenthesis expressions).

$$C_n = \binom{2n}{n} - \binom{2n}{n+1} = \frac{1}{n+1}\binom{2n}{n}$$

A commonly used combinatorial proof for the Catalan numbers involves counting the number of lattice (grid) paths from $(0,0)$ to (n,n) that do not cross above the diagonal $y=x$. Each such path consists of n rightward steps and n upward steps, and the Catalan number counts the number of these "Dyck paths" that never go above the diagonal.

- **Stirling Numbers of the Second Kind:** The number of ways to partition a set of n labeled objects into k non-empty unlabeled subsets. Denoted by $S(n,k)$ or

$$\{n\ k\}$$

.

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

The Stirling numbers of the second kind can also be computed recursively:

$$S(n,k) = k \cdot S(n-1,k) + S(n-1,k-1)$$

with the boundary conditions:

$$S(0,0) = 1; \quad S(n,0) = 0 \text{ for } n > 0; \quad S(0,k) = 0 \text{ for } k > 0$$

- **Bell Number:** The Bell number B^n counts the total number of ways to partition a set of n labeled elements into any number (from 1 up to n) of non-empty, unlabeled subsets. It can also be written as a recurrence relation

$$B^n = \sum_{k=0}^{n} S(n,k)$$

- **Pigeonhole Principle:** If n items are put into m boxes, with $n > m$, then at least one box must contain more than one item.

8.2 Convolutions

8.2.1 AND convolution

$$c[k] = \sum_{i\&j=k} a[i] \cdot b[j]$$

```
vector<mint> and_conv(vector<mint> a, vector<mint> b){
  int n = a.size(); // must be pow of 2
  for (int j = 1; j < n; j <<= 1) {
    for (int i = 0; i < n; i++) {
      if (i&j) {
        a[i^j] += a[i];
        b[i^j] += b[i];
      }
    }
  }

  for (int i = 0; i < n; i++) a[i] *= b[i];

  for (int j = 1; j < n; j <<= 1)
    for (int i = 0; i < n; i++)
      if (i&j) a[i^j] -= a[i];
  return a;
}
```

8.2.2 GCD convolution

$c[k] = \sum_{\gcd(i,j)=k} a[i] \cdot b[j]$

```
vector<mint> gcd_conv(vi a, vi b){
  int n = (int)max(a.size(), b.size());
  a.resize(n); b.resize(n);
  vector<mint> c(n);
  for (int i = 1; i < n; i++) {
    mint x = 0;
    mint y = 0;
    for (int j = i; j < n; j += i) {
```

```
      x += a[j];
      y += b[j];
    }
    c[i] = x*y;
  }
  for (int i = n-1; i >= 1; i--)
    for (int j = 2*i; j < n; j += i)
      c[i] -= c[j];
  return c;
}
```

8.2.3 LCM convolution

$c[k] = \sum_{\text{lcm}(i,j)=k} a[i] \cdot b[j]$

```
vector<mint> lcm_conv(vi a, vi b){
  int n = (int)max(a.size(), b.size());
  a.resize(n); b.resize(n);
  vector<mint> c(n), x(n), y(n);
  for (int i = 1; i < n; i++) {
    for (int j = i; j < n; j += i) {
      x[j] += a[i];
      y[j] += b[i];
    }
    c[i] = x[i]*y[i];
  }
  for (int i = 1; i < n; i++)
    for (int j = 2 * i; j < n; j += i)
      c[j] -= c[i];
  return c;
}
```

8.2.4 OR convolution

$$c[k] = \sum_{i|j=k} a[i] \cdot b[j]$$

```
vector<mint> or_conv(vector<mint> a, vector<mint> b){
  int n = a.size(); // must be pow of 2
  for (int j = 1; j < n; j <<= 1) {
    for (int i = 0; i < n; i++) {
      if (i&j) {
        a[i] += a[i^j];
        b[i] += b[i^j];
      }
    }
  }

  for (int i = 0; i < n; i++) a[i] *= b[i];

  for (int j = 1; j < n; j <<= 1)
    for (int i = 0; i < n; i++)
      if (i&j) a[i] -= a[i^j];
  return a;
}
```

8.2.5 Subset Convolution

8.2.6 OR convolution

$$c[k] = \sum_{i|j=k, i\&j=0} a[i] \cdot b[j]$$

Complexity: $\mathcal{O}(k^2 2^k)$, where k is the number of bits.

```
vector<mint> subset_conv(const vector<mint>& a, const
    vector<mint>& b) {
  int n = a.size(); // must be pow of 2
  int k = __builtin_ctz(n);

  vector<vector<mint>> fa(k+1, vector<mint>(n, 0));
  vector<vector<mint>> fb(k+1, vector<mint>(n, 0));
  vector<vector<mint>> fc(k+1, vector<mint>(n, 0));

  for (int i = 0; i < n; i++) {
    int pc = __builtin_popcount(i);
    fa[pc][i] = a[i];
    fb[pc][i] = b[i];
  }

  for (int pc = 0; pc <= k; pc++) {
    for (int j = 1; j < n; j <<= 1) {
      for (int i = 0; i < n; i++) {
        if (i & j) {
          fa[pc][i] += fa[pc][i ^ j];
          fb[pc][i] += fb[pc][i ^ j];
        }
      }
    }
  }

  for (int i = 0; i < n; i++) {
    for (int pc = 0; pc <= k; pc++) {
      for (int j = 0; j <= pc; j++) {
        fc[pc][i] += fa[j][i] * fb[pc - j][i];
      }
    }
  }

  for (int pc = 0; pc <= k; pc++) {
    for (int j = 1; j < n; j <<= 1) {
      for (int i = 0; i < n; i++) {
        if (i & j) {
          fc[pc][i] -= fc[pc][i ^ j];
        }
      }
    }
  }

  // 5. Gather results
  vector<mint> c(n);
  for (int i = 0; i < n; i++) {
    c[i] = fc[__builtin_popcount(i)][i];
  }

  return c;
}
```

8.2.7 XOR convolution

$$c[k] = \sum_{i \oplus j=k} a[i] \cdot b[j]$$

```
void fwht(vector<mint> &a, bool inv){
  int n = a.size(); // must be pow of 2
  for (int step = 1; step < n; step <<= 1){
    for (int i = 0; i < n; i += 2*step) {
      for (int j = i; j < i+step; j++){
        auto u = a[j];
        auto v = a[j+step];
        a[j] = u+v;
        a[j+step] = u-v;
      }
    }
  }
  if (inv) for (auto &x : a) x /= n;
}

vector<mint> xor_conv(vector<mint> a, vector<mint> b){
  int n = a.size();
  fwht(a,0), fwht(b, 0);
  for (int i = 0; i < n; i++) a[i] *= b[i];
  fwht(a,1);
  return a;
}
```

8.3 Extended Euclid

Time: $\mathcal{O}(\log n)$.

```
int extended_gcd(int a, int b, int &x, int &y) {
  x = 1, y = 0;
  int x1 = 0, y1 = 1;
  while (b) {
    int q = a / b;
    tie(x, x1) = make_tuple(x1, x - q * x1);
    tie(y, y1) = make_tuple(y1, y - q * y1);
    tie(a, b) = make_tuple(b, a - q * b);
  }
  return a;
}
```

8.4 Factorization

Time: $\mathcal{O}(\sqrt{n})$

```
// O(sqrt(n)) fatores repetidos
vi fatora(int n) {
  vi factors;
  for (int x = 2; x*x <= n; x++) {
    while (n%x == 0) {
      factors.push_back(x);
      n /= x;
    }
  }
  if (n > 1) factors.push_back(n);
  return factors;
}
```

8.5 FFT - Fast Fourier Transform

Divide and conquer algorithm used for convolutions and polynomial multiplication. Vector size `a` is a power of 2. Time: $\mathcal{O}(n \log n)$ Space: $\mathcal{O}(n)$

```
void fft(vector<cd> &a, bool invert){
  int len = a.size();
  for(int i = 1, j = 0; i < len; i++){
    int bit = len >> 1;
    while(bit & j){
      j ^= bit;
      bit >>= 1;
    }
    j ^= bit;
    if(i < j) swap(a[i], a[j]);
  }
  for(int l = 2; l <= len; l <<= 1){
    double ang = 2*PI/l * (invert ? -1: 1);
    cd wd(cos(ang), sin(ang));
    for(int i = 0; i < len; i += l){
      cd w(1);
      for(int j = 0; j < l/2; j++){
        cd u = a[i+j], v = a[i+j+l/2];
        a[i+j] = u+w*v;
        a[i+j+l/2] = u-w*v;
        w *= wd;
      }
    }
  }
  if(invert){
    for(int i = 0; i < len; i++){
      a[i] /= len;
    }
  }
}
```

8.6 Inclusion-Exclusion Principle

For a set of k primes, count numbers from 1 to n divisible by at least one prime.

For every non-empty subset of primes:

- If the subset has odd size, add $n/prod$.
- If the subset has even size, subtract $n/prod$.
- $prod$ is the product of the selected primes.

Here, $n/prod$ is the number of integers from 1 to n divisible by all selected primes.

Complexity: $\mathcal{O}(2^k k)$.

```
int n;
vi primes;
int ans = 0;

for (int mask = 1; mask < (1 << primes.size()); mask++)
    {
  int prod = 1;
  for (int bit = 0; bit < primes.size(); bit++) {
    if (mask>>bit&1) {
      prod *= primes[bit];
      if (prod > n) break;
    }
  }
  if (__popcount(mask) & 1) ans += n/prod;
  else ans -= n/prod;
}
```

8.7 Legendre's formula

Computes the largest power of a prime p in n!

```
int legendre(int n, int p){
  int ans = 0;
  while(n>0){
    n /= p; ans += n;
  }
  return ans;
}
```

8.8 Linear Sieve + Möbius Function

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1 \\ (-1)^k & \text{if } n \text{ is a product of } k \text{ distinct primes} \\ 0 & \text{if } p^2 \mid n \text{ for some prime } p \end{cases}$$

8.8.1 Applications

- **Inclusion-Exclusion:** Naturally generates $1, -1, 0$ coefficients for prime factor sets.
- **Möbius Inversion:** Converts $g(n) = \sum_{d|n} f(d)$ to $f(n) = \sum_{d|n} \mu(d) g(\frac{n}{d})$. Often reduces $O(N)$ divisor sums to $O(\sqrt{N})$.
- **Dirichlet Convolution:** $\mu * \mathbf{1} = \epsilon$. Core to sub-linear prefix sum algorithms (e.g., DuSieve) yielding $O(N^{2/3})$ time.

```
auto sieve = [](int n) -> pair<vi,vi> {
  vi primes, mu(n+1);
  mu[1] = 1;
  vector<bool> is_prime(n+1, true);
  is_prime[0] = is_prime[1] = false;
  for (int i = 2; i <= n; ++i) {
    if (is_prime[i]) {
      primes.push_back(i);
      mu[i] = -1;
    }
    for (int p : primes) {
      if (i*p > n) break;
      is_prime[i*p] = false;
      if (i%p == 0) {
        mu[i*p] = 0;
        break;
      }
      mu[i*p] = -mu[i];
    }
  }
  return {primes, mu};
};
```

8.9 Matrix template

A template for square matrices, used for solving linear recurrences with fast exponentiation, memory is on a 1D vector, but can be accessed with `A[i][j]` normally (custom `operator[]`)

```
template<typename T>
struct mat{
  vector<T> m;
  int n;

  mat(int _n = 0, bool identity = false) : n(_n) {
    m.resize(n*n);
    if (!identity) return;
    for (int i = 0; i < n; i++)
      m[i*n+i] = 1;
  }

  mat& operator+=(const mat &o){
    for (int i = 0; i < n; i++){
      int ra = i*n;
      for (int j = 0; j < n; j++){
        m[ra+j] += o.m[ra+j];
      }
    }
    return *this;
  }
  mat& operator-=(const mat& o){
    for (int i = 0; i < n; i++){
      int ra = i*n;
      for (int j = 0; j < n; j++){
        m[ra+j] -= o.m[ra+j];
      }
    }
    return *this;
  }
  mat& operator*=(const mat& o){
    vector<T> ans(n*n);
    for (int i = 0; i < n; i++){
      for (int k = 0; k < n; k++){
        int ra = i*n, rb = k*n;
        for (int j = 0; j < n; j++){
          ans[ra+j] += m[ra+k] * o.m[rb+j];
        }
      }
    }
    this->m = ans;
    return *this;
  }
  friend mat operator+(mat a, const mat& b){
    return a+=b;
  }
  friend mat operator-(mat a, const mat& b){
    return a-=b;
  }
  friend mat operator*(mat a, const mat& b){
    return a*=b;
  }
  T* operator[](int i){
    return &m[i*n];
  }
  vector<T> operator*(const vector<T> &v){
    vector<T> ans(n);
    for (int i = 0; i < n; i++){
      int ra = i*n;
      for (int j = 0; j < n; j++)
        ans[i] += m[ra+j]*v[j];
```

```
    }
    return ans;
  }
  mat operator^(int e){
    return exp(*this, e);
  }
  static mat exp(mat b, ll e){
    mat ans = mat(b.n,true);
    while(e>0){
      if(e&1) ans*=b;
      b*=b;
      e>>=1;
    }
    return ans;
  }
};
```

8.10 Mint

```
template<const int MOD>
struct mint {
  int val;
  mint(ll v=0){
    ll x = v%MOD;
    if (x<0) x+=MOD;
    val = x;
  }
  mint& operator+=(const mint &o){
    ll x = (ll)val + o.val;
    if (x >= MOD) x -= MOD;
    val = x;
    return *this;
  }
  mint& operator-=(const mint &o){
    ll x = (ll)val - o.val;
    if (x < 0) x += MOD;
    val = x;
    return *this;
  }
  mint& operator*=(const mint &o){
    val = (1ll*val*o.val)%MOD;
    return *this;
  }
  mint& operator/=(const mint &o){
    val = (1ll*val*inv(o).val)%MOD;
    return *this;
  }
  friend mint operator+(mint a, const mint &b) { return a+=b; }
  friend mint operator-(mint a, const mint &b) { return a-=b; }
  friend mint operator*(mint a, const mint &b) { return a*=b; }
  friend mint operator/(mint a, const mint &b) { return a/=b; }
  static mint power(mint b, int e){
    mint ans = 1;
    while(e>0){
      if (e&1) ans *= b;
      b*=b; e>>=1;
    }
    return ans;
  }
  static mint inv(mint n){ return power(n,MOD-2); }
};
```

8.11 Modular Inverse

If m is prime, can use binary exponentiation to compute a^{p-2} (Fermat's Little Theorem).

This code works for non-prime m, as long as it is coprime to a.

Time: $\mathcal{O}(\log m)$

```
int modInverse(int a, int m) {
  int x, y;
  int g = extendedGcd(a, m, x, y);
  if (g != 1) return -1;
  return (x % m + m) % m;
}
```

8.12 Number Theoretic Transform (NTT)

NTT is a fast algorithm (analogous to FFT) for polynomial multiplication modulo a special prime. It requires a prime modulus $p = c \cdot 2^k + 1$ (a "primitive root prime") and a primitive 2^k-th root of unity modulo p.

- **Prime Choices:** To use NTT, pick a modulus and a matching primitive root (see table below). For arbitrary moduli (e.g., $10^9 + 7$), multiply with several NTT-friendly primes and reconstruct with CRT (see `crt_multiply`).
- **Time Complexity:** $\mathcal{O}(n \log n)$ for polynomial multiplication.

8.12.1 NTT-Friendly Primes and Roots

NTT-friendly primes and their primitive roots:

- Mod: 998244353, Root: 3, Max N: 2^{23}
- Mod: 734003201, Root: 3, Max N: 2^{20}
- Mod: 167772161, Root: 3, Max N: 2^{25}
- Mod: 469762049, Root: 3, Max N: 2^{26}

Use the modulus as `MOD` and the root as `ROOT` when instantiating the NTT.

- For large/concrete moduli, see `crt_multiply` in the code for a multi-modulus solution with Chinese Remainder Theorem (CRT).

```
namespace NTT{
  template<typename T, ll MOD, ll ROOT>
  void transform(vector<T>& a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
      int bit = n >> 1;
      for (; j & bit; bit >>= 1) j ^= bit;
      j ^= bit;
      if (i < j) swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
      T wlen = T::power(ROOT, (MOD - 1) / len);
      if (invert) wlen = T::inv(wlen);
      for (int i = 0; i < n; i += len) {
        T w = 1;
        for (int j = 0; j < len / 2; j++) {
          T u = a[i + j], v = a[i + j + len / 2] * w;
          a[i + j] = u + v;
          a[i + j + len / 2] = u - v;
          w *= wlen;
        }
      }
    }

    if (invert) {
      T n_inv = T::inv(n);
      for (T& x : a) x *= n_inv;
    }
  }

  template<typename T, ll MOD, ll ROOT>
  vector<ll> multiply(const vector<ll>& a, const vector<ll>& b) {
    vector<T> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size()) n <<= 1;
    fa.resize(n);
    fb.resize(n);

    transform<T, MOD, ROOT>(fa, false);
    transform<T, MOD, ROOT>(fb, false);

    for (int i = 0; i < n; i++) fa[i] *= fb[i];

    transform<T, MOD, ROOT>(fa, true);

    vector<ll> result(n);
    for (int i = 0; i < n; i++) result[i] = fa[i].val;
    return result;
  }

  vector<ll> crt_multiply(const vector<ll>& a, const vector<ll>& b) {
    const ll mod1 = 998244353;
    const ll root1 = 3;
    using mint1 = mint<mod1>;
    vector<ll> ans1 = NTT::multiply<mint1, mod1, root1>(a, b);

    const ll mod2 = 1004535809;
    const ll root2 = 3;
    using mint2 = mint<mod2>;
    vector<ll> ans2 = NTT::multiply<mint2, mod2, root2>(a, b);

    int ans_size = a.size() + b.size() - 2;
    ll M1_inv_M2 = mint<mod2>::inv(mod1).val;

    vector<ll> final_result(ans_size + 1);
    for (int i = 0; i <= ans_size; ++i) {
      ll v1 = ans1[i];
      ll v2 = ans2[i];
      ll k = ((v2 - v1 + mod2) % mod2 * M1_inv_M2) % mod2;
```

```
        final_result[i] = v1 + k * mod1;
    }
    return final_result;
  }
}
```

8.13 Euler's Totient

Returns the amount of numbers smaller than n that are coprime to n. Time: $\mathcal{O}(\sqrt{n})$

```
int phi(int n){
  int ans = n;
  for (int i = 2; i*i <= n; i++){
    if (n%i == 0){
      while(n%i == 0) n/=i;
      ans -= ans/i;
    }
  }
  if (n>1) ans -= ans/n;
  return ans;
}
```

9 Geometry

9.1 Convex hull - Graham Scan

Time: $\mathcal{O}(n \log n)$

```
#define CLOCKWISE -1
#define COUNTERCLOCKWISE 1
#define INCLUDE_COLLINEAR 0 // pode mudar

struct Point {
    ll x, y;
    bool operator==(Point const& t) const {
        return x == t.x && y == t.y;
    }
};

struct Vec {
    int x, y, z;
};

Vec cross(Vec v1, Vec v2){
    int x = v1.y*v2.z - v1.z*v2.y;
    int y = -v1.x*v2.z + v1.z*v2.x;
    int z = v1.x*v2.y - v1.y*v2.x;
    return {x,y,z};
}

ll dist2(Point p1, Point p2){
    int dx = p1.x-p2.x;
    int dy = p1.y-p2.y;
    return dx*dx+dy*dy;
}

ll orientation(Point pivot, Point a, Point b){
    Vec va = {a.x-pivot.x, a.y-pivot.y, 0};
    Vec vb = {b.x-pivot.x, b.y-pivot.y, 0};
    Vec v = cross(va,vb);
    if (v.z < 0) return CLOCKWISE;
    if (v.z > 0) return COUNTERCLOCKWISE;
    return 0;
}

bool clock_wise(Point pivot, Point a, Point b) {
    int o = orientation(pivot, a, b);
    return o < 0 || (INCLUDE_COLLINEAR && o == 0);
}

bool collinear(Point a, Point b, Point c) { return orientation(a, b, c) == 0; }

vector<Point> convex_hull(vector<Point> &points, bool counterClockwise) {
    int n = points.size();
    Point pivot = *min_element(points.begin(), points.end(), [](Point a, Point b) {
        return ii(a.y, a.x) < ii(b.y, b.x);
    });

    sort(points.begin(), points.end(), [&](Point a, Point b) {
        int o = orientation(pivot, a, b);
        if (o == 0) return dist2(pivot, a) < dist2(pivot, b);
        return o == CLOCKWISE;
    });

    if (INCLUDE_COLLINEAR) {
        int i = n-1;
        while (i >= 0 && collinear(pivot, points[i], points.back())) i--;
        reverse(points.begin()+i+1, points.end());
    }

    vector<Point> hull;
    for (auto p : points) {
        while (hull.size() > 1 && !clock_wise(hull[hull.size()-2], hull.back(), p))
            hull.pop_back();
        hull.push_back(p);
    }
    if (!INCLUDE_COLLINEAR && hull.size() == 2 && hull[0] == hull[1])
        hull.pop_back();

    if (counterClockwise && hull.size() > 1) {
        vector<Point> reversed_hull = hull;
        reverse(reversed_hull.begin() + 1, reversed_hull.end());
        return reversed_hull;
    }
    return hull;
}
```

9.2 Basic elements - geometry lib

- Basic elements for using the geometry lib, contains points, vector operations and distances between points, distance between point and segment, distance between segments, segment intersection check, orientation check (ccw).
- Always use long double for floating point. Only use floating point if indispensable.
- For a == b, use |a - b| < eps.!!!!

Time: $\mathcal{O}(1)$

9.2.1 Polygon Area

- **Heron's Formula for triangle area**:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

, where a, b, and c are the triangle sides and $s = (a+b+c)/2$

TODO Shoelace

- **Pick's Theorem for polygon area with integer coordinates**:

$$A = a + b/2 - 1$$

, where a is the number of integer coordinates inside the polygon and b is the number of integer coordinates on the polygon boundary. b can be calculated for each edge as

$$b = gcd(xi + 1 - xi, yi + 1 - yi) + 1$$

.

Polygon Area Time: $\mathcal{O}(n)$

9.2.2 Point in polygon

Sum of edge angles relative to the point must sum to 2pi
Time: $\mathcal{O}(n \log n)$

```
#include<bits/stdc++.h>
using namespace std;
typedef long double ld;
#define eps 1e-9
#define pi 3.141592653589
#define int long long int


struct pt {
    int x, y;
    int operator==(pt b) {
        return x == b.x && y == b.y;
    }
    int operator<(pt b) {
        if(x == b.x) return y < b.y;
        return x < b.x;
    }
    pt operator-(pt b) {
        return {x - b.x, y - b.y};
    }
    pt operator+(pt b) {
        return {x+b.x, y + b.y};
    }
};
int cross(pt u, pt v) {
    return u.x * v.y - u.y * v.x;
```

```
}
int dot(pt u, pt v) {
    return u.x * v.x + u.y * v.y;
}
ld norm(pt u) {
    return sqrt(dot(u, u));
}
ld dist(pt u, pt v) {
    return norm(u - v);
}
int ccw(pt u, pt v) { // cuidado com colineares!!!!
    return (cross(u, v) > eps)?1:((fabs(cross(u, v)) < eps)?0:-1);
}
int pointInSegment(pt a, pt u, pt v) { // checks if a lies in uv
    if(ccw(v - u, a - u)) return 0;
    vector<pt> pts = {a, u, v};
    sort(pts.begin(), pts.end());
    return pts[1] == a;
}
ld angle(pt u, pt v) { // angle between two vectors
    ld c = cross(u, v);
    ld d = dot(u,v);
    return atan2l(c, d);
}
int intersect(pt sa, pt sb, pt ra, pt rb) {// not sure if it works when one of the segments is a point
    pt s = sb - sa, r = rb - ra;
    if(pointInSegment(sa, ra, rb) || pointInSegment(sb, ra, rb) ||pointInSegment(ra, sa, sb) || pointInSegment(rb, sa, sb)) return 1;
    return !(ccw(s, ra - sa) == ccw(s, rb - sa) || ccw(r, sa - ra) == ccw(r, sb - ra));
}

ld polygonArea(vector<pt>& p) {// not signed (for signed area remove the absolute value at the end)
    ld area = 0;
    int n = p.size() - 1; // p[n] = p[0]
    for(int i = 0; i < n; i++) {
        area += cross(p[i], p[i + 1]);
    }
    return fabs(area)/2;
}
int pointInPolygon(pt a, vector<pt>& p) {// returns 0 for point in BOUNDARY, 1 for point in polygon and -1 for outside
    ld total = 0;
    int n = p.size() - 1;
    for(int i = 0; i < n; i++) {
        pt u = p[i] - a;
        pt v = p[i + 1] - a;
        if(fabs(dist(p[i], a) + dist(p[i + 1], a) - dist(p[i], p[i + 1])) < eps) {
            return 0;
        }
        total += angle(u, v);
    }

    return (fabs(fabs(total) - 2 * pi) < eps)?1:-1;
}



signed main() {
    int n, m; scanf("%lld %lld", &n, &m);
    vector<pt> p(n + 1);
    for(int i = 0; i < n; i++) {
        scanf("%lld %lld", &p[i].x, &p[i].y);
    }
    p[n] = p[0];

    while(m--) {
        pt a; scanf("%lld %lld", &a.x, &a.y);
        int ans = pointInPolygon(a, p);
        printf("%s\n", (ans > 0)?"INSIDE":(ans?"OUTSIDE":"BOUNDARY"));
    }
}
```